Robustness and Verification Limits of a Hybrid DWT–DCT–SVD Image Watermark: A 100-Image Evaluation with False-Positive Controls

Krishang Vikram Singh
Roll No. B106 · Batch C1 · B.Tech Computer Engineering
NMIMS Mukesh Patel School of Technology Management & Engineering (MPSTME), Mumbai, India

Abstract—Singular-value watermarking in a wavelet–cosine transform domain is widely used for ownership marking, yet reported evaluations typically use a handful of images and score only the correlation between the extracted and the owner's logo. We re-evaluate a classical hybrid DWT–DCT–SVD scheme on 100 unique 512×512 images (94 real, 6 labelled synthetic) under 14 attacks plus an unattacked reference (3000 seeded trials), with an embedding-strength sweep, false-positive controls, a comparison with an open-source blind block-based hybrid and two classical baselines, and an exploratory per-frame video test. The scheme is imperceptible (mean PSNR 36.0 dB, SSIM 0.992) and recovers the logo (NC ≥ 0.75) in 85.1% of trials: ≥ 95 % of images survive JPEG (Q ≥ 20), median filtering, sharpening, rescaling and mild noise, but only 72% survive 15° rotation, 44% histogram equalisation and 1% border cropping. Two findings qualify common practice. First, the robustness–imperceptibility trade-off in the embedding strength α is flat for JPEG and appears only under harsher attacks. Second, a different logo decoded with the owner's key still yields NC 0.975 (genuine pair 0.977), and 10%–13% of unrelated-image controls exceed the customary NC 0.75 threshold: a high NC does not show that the claimed watermark is present. The blind block-based hybrid achieves lower bit-error rates under most attacks but is 13× slower and weaker under rotation. Code, data manifest and all results accompany the paper.

Index Terms—*digital watermarking, DWT, DCT, SVD, robustness, false-positive problem, normalised correlation, PSNR, SSIM.*

I. INTRODUCTION

Images are copied at no cost, so owners embed invisible signatures that can later be extracted to support an ownership claim [1], [2]. Spatial-domain marks such as least-significant-bit (LSB) substitution are fragile [3], whereas transform-domain schemes place the mark in perceptually significant spectral components [4]. SVD-based schemes for ownership protection were introduced by Liu and Tan [5], combined with the wavelet transform by Ganic and Eskicioglu [6], and later cascaded with the DCT in hybrid DWT–DCT–SVD designs [7], [8].

Two gaps motivate this study. First, evaluations of such schemes usually rely on a few standard images and report one number, the normalised correlation (NC) with the owner's logo. Second, SVD schemes that store the logo's singular vectors in the key have long been criticised for false positives and ambiguity attacks [9], [10], but the effect is rarely quantified, so the common NC ≥ 0.75 rule is used without knowing its false-alarm rate.

Contributions. (1) A seeded, fully logged evaluation of a classical DWT–DCT–SVD scheme on 100 unique images with 14 attacks and an unattacked reference. (2) Control experiments that measure the false-positive problem and a metric analysis (a blank image already scores NC 0.36 against our logo; a mean-centred variant, ZNCC, is added). (3) An embedding-strength study showing that the trade-off depends on the attack, correcting a single-attack plot. (4) A measured comparison with an existing open-source blind block-based hybrid [11] and two classical baselines, and an exploratory per-frame video test.

II. RELATED WORK

Foundations. Cox *et al.* [4] argued that a watermark must reside in perceptually significant coefficients and be detected by correlation; Koch and Zhao [12] tie block-DCT embedding to the JPEG model; Hartung and Kutter [1] survey the field. The DWT [13] and DCT [14] underlie JPEG-family codecs [15]. **SVD schemes.** Liu and Tan [5] modify singular values for rightful-ownership protection; Ganic and Eskicioglu [6] embed in the wavelet sub-bands; Lai and Tsai [16] and later hybrids [7], [8] report improved robustness (we did not re-evaluate those claims). **Critique.** Zhang and Li [9] commented on [5], and Ting [10] studied ambiguity attacks on [6]: when the logo's singular vectors are part of the key, the detector can reconstruct the key owner's logo regardless of what was embedded. **Geometry and video.** Lin *et al.* [17] show that rotation, scale and translation defeat correlation detectors unless the detection domain is invariant; Hartung and Girod [18] treat compressed video. Learned watermarking [19], [20] is outside our scope.

III. METHOD

A. Embedding and extraction

A one-level Haar DWT splits the 512×512 host into four sub-bands; the 256×256 approximation LL is transformed by a global orthonormal 2-D DCT (not block-wise) and factorised by SVD, $A = U \cdot \mathrm{diag}(S) \cdot V^{T}$. The 64×64 binary logo W is resized to the sub-band size and factorised, $W = U_w \cdot \mathrm{diag}(S_w) \cdot V_w^{T}$. With embedding strength α (0.10 unless stated),

$$S' = S + \alpha \cdot S_w , \quad A' = U \cdot \mathrm{diag}(S') \cdot V^{T} \qquad (1)$$

followed by the inverse DCT, the inverse DWT and clipping to [0, 255]. The key is $K = \{S_orig, U_w, V_w^{T}, \alpha\}$. Given a (possibly attacked) image, the same transforms give singular values S_att, and

$$S_w' = (S_att - S_orig)/\alpha , \quad W' = U_w \cdot \mathrm{diag}(S_w') \cdot V_w^{T} \qquad (2)$$

The scheme is therefore **non-blind** (S_orig is derived from the unmarked host), and the logo's identity is carried by the key (U_w, V_w), not by the image — the structural reason for the false-positive behaviour measured in Section V-E. The embedding and extraction code is unchanged from the first implementation of this project; numerical results of that implementation were reproduced to within 0.004 NC.

B. Attacks and protocol

Fourteen attacks plus the unattacked reference: JPEG at quality 90/50/20; Gaussian noise ($\sigma^2 = 0.005$, 0.02 on [0,1]); salt-and-pepper (2 %); 3×3 median; Gaussian blur ($\sigma = 1.5$); unsharp-mask sharpening; rotation by 5° and 15°; border zeroing with 25 % of each dimension removed (43.75 % of the area); 0.5× down/up-scaling; and histogram equalisation. The watermarked image is rounded to 8 bit before attacking (primary protocol); attacking the floating-point image, as in the first implementation, changed mean NC by less than 0.001 except for equalisation (0.644 vs 0.638). All stochastic attacks are seeded.

C. Metrics

Imperceptibility is measured against the unwatermarked original by PSNR = $10 \log_{10}(255^2/\mathrm{MSE})$ and SSIM [21]. Recovery is measured by $\mathrm{NC} = \Sigma\, W \cdot W' / (\|W\| \|W'\|)$, by BER after thresholding at 127.5, and by ZNCC, the NC of the mean-centred images. For this logo a blank white image scores NC = 0.359 and a rank-1 approximation 0.868, so NC alone flatters bright outputs. A logo is called recovered if NC ≥ 0.75, a convention we keep for comparability with earlier work; we could not trace it to a primary source and estimate its false-alarm rate in Section V-E.

IV. EXPERIMENTAL SETUP

Data. 100 unique images (94 real, 6 synthetic controls: gradients, grating, noise, checkerboard) were drawn from open sample collections hosted on GitHub (OpenCV, scikit-image, three.js textures and others), converted to grey (ITU-R BT.601), centre-cropped and resized to 512×512; near-duplicates were removed by a difference hash. The set is a convenience sample covering animals, urban scenes, objects, textures, portraits, paintings, documents, astronomy and medical images with dark/bright and low/high-contrast cases; it is not a recognised benchmark, and per-image provenance, licences and rights notes are recorded in the metadata.

Experiments. E1 imperceptibility (100 images); E2 robustness (3000 trials = 100 images × 15 conditions × 2 protocols); E3 an α sweep over eight values and four attacks (4000 rows); E4 false-positive controls (1600 rows): unmarked and marked *other* images decoded with the key of image *i*, and the same host marked with a *different* logo decoded with the key of logo A. S1 compares the scheme, at equal PSNR and with an identical 4095-bit payload, with `blind-watermark` 0.4.4 [11] (blind, block-based DWT–DCT–SVD), an 8×8 block-DCT baseline and an LSB baseline on 30 random real images. V1 applies the scheme frame by frame to two real clips.

Statistics and reproducibility. Means carry 95 % bootstrap intervals, pass rates Wilson intervals, associations Spearman correlations. Two complete executions gave identical metrics (maximum absolute difference 0.0); 43 unit tests pass. Software: NumPy [22], SciPy [23], scikit-image [24], PyWavelets [25], Pillow, OpenCV and FFmpeg; four vCPUs, single-thread BLAS.

V. RESULTS AND DISCUSSION

A. Imperceptibility

The mean PSNR of the stored 8-bit watermarked image is 35.99 dB (range 35.5–43.8; none below 30 dB) and the mean SSIM is 0.992 (Fig. 1). PSNR is almost host-independent (82 of 100 images lie in 35.5–35.75 dB) because the added energy is set by the logo's singular values. SSIM exposes exceptions: the near-black astronomy image reaches 35.9 dB but SSIM 0.58, and three images fall below 0.98; PSNR alone would have hidden this.



Fig. 1. Original, watermarked and amplified difference (image_066, α = 0.10).

B. Robustness

TABLE I
Robustness over 100 images per condition (8-bit protocol).

Attack	mean NC (95 % CI)	≥0.75 (%)	ZNCC	BER
No Attack	0.977 (0.972–0.980)	100	0.974	0.009
JPEG Q=90	0.977 (0.972–0.979)	100	0.974	0.009
JPEG Q=50	0.975 (0.971–0.979)	100	0.972	0.009
JPEG Q=20	0.956 (0.932–0.975)	97	0.949	0.009
Gauss. Noise (var=0.005)	0.922 (0.884–0.952)	95	0.911	0.015
Gauss. Noise (var=0.02)	0.806 (0.755–0.852)	86	0.778	0.041
Salt & Pepper (2%)	0.928 (0.900–0.950)	97	0.919	0.018
Median Filter 3x3	0.967 (0.960–0.973)	100	0.964	0.008
Gauss. Blur (sigma=1.5)	0.913 (0.890–0.932)	96	0.897	0.022
Rotation 5 deg	0.880 (0.848–0.907)	92	0.860	0.037
Rotation 15 deg	0.776 (0.733–0.818)	72	0.739	0.067
Cropping 25%	0.022 (0.006–0.045)	1	-0.036	0.140
Scaling 0.5x	0.938 (0.916–0.953)	98	0.926	0.017
Hist. equalisation	0.644 (0.599–0.688)	44	0.615	0.126
Sharpening	0.966 (0.954–0.975)	99	0.963	0.009

1277 of 1500 trials (85.1%) reach NC ≥ 0.75 including the 100 unattacked references, as counted in earlier work; counting only the 14 attacks it is 84.1% (mean NC 0.834). ZNCC is about 0.02 lower than NC and yields 84.3% overall, so the conclusions are not an artefact of the brightness offset. Table I and Fig. 2 show strong robustness to compression and filtering and failure under cropping and equalisation. Rotation is weaker than sometimes assumed: mean NC 0.880 at 5° and 0.776 at 15° (92 % and 72 % pass). The scheme's robustness to JPEG stems from the SVD stage; the DCT is global and not aligned with JPEG's 8×8 blocks.

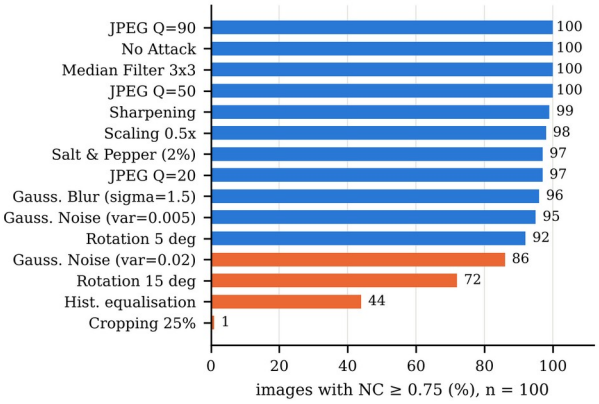


Fig. 2. Share of the 100 images with NC ≥ 0.75 per attack.

C. Which images fail

Robustness is image-dependent. NC after attack is negatively correlated with image contrast (Spearman ρ: salt-and-pepper -0.68, Gaussian noise σ^2=0.02 -0.65, 15° rotation -0.55); histogram equalisation instead tracks mean brightness (ρ = -0.31). Document-text images are the weakest category under noise (mean NC 0.53). These are associations with uncorrected p-values (n = 100), not demonstrated causes.

D. Embedding strength

Mean PSNR falls from 49.1 dB (α = 0.02) to 36.0 dB (0.10) and 24.2 dB (0.40). After JPEG Q = 50 the mean NC is essentially flat in α (0.960 at 0.02, 0.973 at 0.40). The trade-off appears under harsher attacks: for Gaussian noise (σ^2 = 0.02) the pass rate rises from 9% at α = 0.02 to 85% at 0.10 and 95% at 0.40 while PSNR drops below 30 dB for α ≥ 0.20 (Fig. 3). A single-attack plot on a second, zoomed NC axis — and a JPEG simulation that truncates floating-point pixels, which lowers NC only at α = 0.02 from 0.981 to 0.929 — had earlier suggested a steep trade-off for JPEG; we reproduced that figure exactly and traced the artefact.

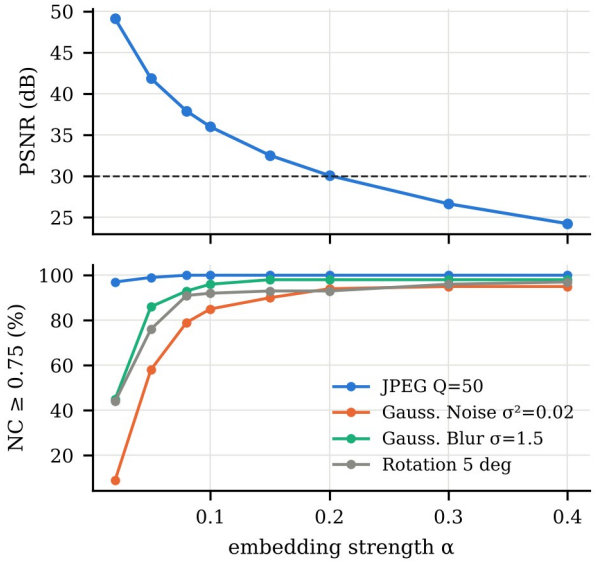


Fig. 3. Effect of α: PSNR (top) and pass rate after four attacks (bottom), 100 images.

E. False-positive controls

TABLE II
False-positive controls (E4).

	Control (NC against the owner's logo)	mean	p95	≥0.75 (%)
ref	genuine: own key, clean image	0.977	0.981	100.0
a	other image, unmarked, key of i	0.368	0.804	9.8
b	other image, marked, wrong key	0.401	0.832	12.6
c	other logo B, decoded with key A	0.975	0.980	100.0

Embedding a *different* logo in the same host and decoding with the key of logo A returns the owner's logo: NC 0.975, indistinguishable from the genuine 0.977 (Fig. 4, 200 trials) — the extractor reproduces the key owner's logo whatever singular values it finds. Unrelated images decoded with a wrong key score 0.37–0.40 on average, with 95th percentile 0.82; 9.8% (unmarked) and 12.6% (marked) exceed 0.75, but only about 1 % exceed 0.90. A detector calibrated to a 5 % false-alarm rate would reject 19% of the attacked genuine trials. Decoding the unmarked host with its own key gives NC = 0 by construction (S_att = S_orig) and must not be read as evidence. An attempt to cure the problem by embedding the logo's principal component made it worse (an unmarked image still decoded at NC 0.98), because the key then contains the whole logo. Claims of ownership therefore require an additional safeguard, for example an image-bound signature, which is future work.

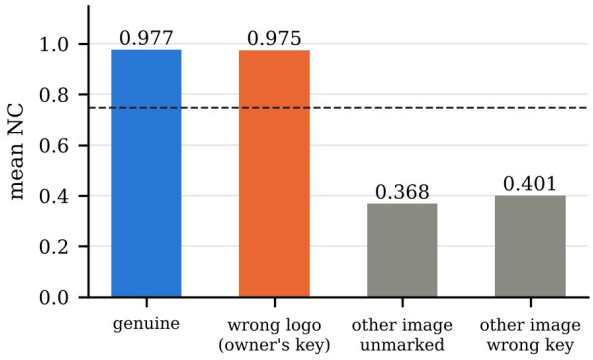


Fig. 4. Mean NC for the genuine pair and for the controls (dashed: 0.75).

F. Comparison with existing technology

TABLE III
Same 4095-bit payload and attacks.

BER (30 images)	Project	blind-wm.	block DCT	LSB
JPEG Q=50	0.010	0.004	0.296	0.426
JPEG Q=20	0.010	0.074	0.469	0.531
Gauss. noise	0.038	0.005	0.182	0.500
Rotation 5°	0.029	0.214	0.493	0.504
Cropping	0.135	0.060	0.124	0.128
Scaling 0.5×	0.015	0.004	0.483	0.455
PSNR (dB)	35.9	42.4	35.1	69.5
embed/extract (ms)	32/17	420/315	26/12	2/1

At equal PSNR the project is far more robust than the LSB and 8×8 block-DCT baselines, which approach chance (BER 0.5) under JPEG, rescaling and rotation (Fig. 5). It is, however, **not** superior to the existing blind block-based hybrid [11]: that package has lower BER under JPEG Q = 50 (0.004 vs 0.010), noise, blur, scaling and cropping at a higher PSNR (42.4 vs 35.9 dB) without a key, but is 13× slower to embed and 18× slower to extract, and worse under rotation (0.214 vs 0.029) and JPEG Q = 20 (0.074 vs 0.010). Its YUV design and our channel-wise attacks may favour it. Library-level checks

showed that DCT, SVD, Haar DWT and JPEG implementations in SciPy, OpenCV, NumPy and Pillow agree to $\leq 1.5 \cdot 10^{-16}$ (relative) or bit-identically, and that OpenCV is faster for attack operations (e.g. 3×3 median ≈ 340×, partly because it works on 8-bit data); the whole experiment runs in about 80 s on four cores, so speed was not a constraint.

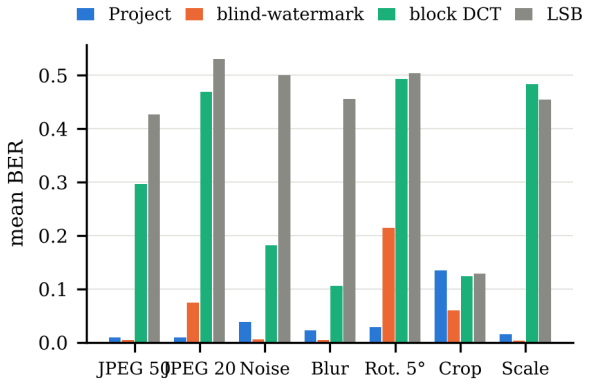


Fig. 5. Mean bit-error rate of four schemes (lower is better; 0.5 = chance).

G. Video feasibility

Applied frame by frame (each frame with its own key) to 60 frames of two real clips and re-encoded with H.264 (CRF 18/23/28) and MJPEG (q 5/15), the logo was recovered in all 720 decoded frames (NC ≥ 0.97); embedding takes about 32–41 ms per frame on one core. This is a feasibility check only: temporal attacks, colour video and hardware codecs were not tested, and a naive frame-drop test with position-aligned keys still gave NC 0.97 because neighbouring frames are nearly identical — a further symptom of key-dependence.

VI. LIMITATIONS

(i) The false-positive problem is measured, not solved. (ii) The scheme is non-blind and needs a key of about 2 MB per image. (iii) Border cropping and rotation beyond 5° defeat it; equalisation is only partly survived. (iv) Grey-scale only; clean extraction is not lossless (BER ≈ 0.9 % from logo resampling). (v) The 94 real images are a convenience sample from GitHub-hosted sets with mostly undocumented photograph rights, standardised to grey squares; category-level results rest on 1–14 images. (vi) Only one existing watermarking package was benchmarked; learned methods [19], [20] and GPU variants were not. (vii) Timing varies by 5–10 % between runs. (viii) The video test covers 120 distinct frames from two clips.

VII. CONCLUSION AND FUTURE WORK

A classical DWT–DCT–SVD watermark is imperceptible (PSNR 36.0 dB, SSIM 0.992) and survives everyday processing — JPEG, median filtering, sharpening, rescaling, mild noise — on at least 95 % of 100 images, but not cropping, equalisation or strong rotation. Larger-scale testing changes two common conclusions: the α trade-off depends on the attack, and a high normalised correlation does not prove that the claimed watermark is present, because a different logo decoded with the owner's key scores almost the same. An existing blind block-based hybrid outperforms the scheme on most attacks, so the scheme's value lies in its transparency and compactness, not in robustness. Future work: bind the logo to the image (signature of image features), block-based redundant embedding for cropping, geometric re-synchronisation [17], colour via luminance, and index-aware keys with temporal redundancy for video [18].

REFERENCES

[1] F. Hartung and M. Kutter, "Multimedia watermarking techniques," Proc. IEEE, vol. 87, no. 7, pp. 1079–1107, Jul. 1999, doi: 10.1109/5.771066.

[2] I. J. Cox, M. L. Miller, J. A. Bloom, J. Fridrich, and T. Kalker, Digital Watermarking and Steganography, 2nd ed. Burlington, MA, USA: Morgan Kaufmann, 2008.

[3] R. G. van Schyndel, A. Z. Tirkel, and C. F. Osborne, "A digital watermark," in Proc. IEEE Int. Conf. Image Process. (ICIP), vol. 2, 1994, pp. 86–90.

[4] I. J. Cox, J. Kilian, F. T. Leighton, and T. Shamoon, "Secure spread spectrum watermarking for multimedia," IEEE Trans. Image Process., vol. 6, no. 12, pp. 1673–1687, Dec. 1997, doi: 10.1109/83.650120.

[5] R. Liu and T. Tan, "An SVD-based watermarking scheme for protecting rightful ownership," IEEE Trans. Multimedia, vol. 4, pp. 121–128, 2002.

[6] E. Ganic and A. M. Eskicioglu, "Robust DWT-SVD domain image watermarking: Embedding data in all frequencies," in Proc. Workshop Multimedia and Security (MM&Sec '04), ACM, 2004, pp. 166–174.

[7] S. Bekkouch and K. M. Faraoun, "Robust and reversible image watermarking scheme using combined DCT-DWT-SVD transforms," J. Inf. Process. Syst., vol. 11, no. 3, pp. 406–420, 2015, doi: 10.3745/JIPS.02.0021.

[8] M. Begum, J. Ferdush, and M. S. Uddin, "A hybrid robust watermarking system based on discrete cosine transform, discrete wavelet transform, and singular value decomposition," J. King Saud Univ. – Comput. Inf. Sci., vol. 34, no. 8, pp. 5856–5867, 2022.

[9] X.-P. Zhang and K. Li, "Comments on 'An SVD-based watermarking scheme for protecting rightful ownership'," IEEE Trans. Multimedia, vol. 7, pp. 593–594, 2005.

[10] G. C.-W. Ting, "Ambiguity attacks on the Ganic–Eskicioglu robust DWT-SVD image watermarking scheme," in Proc. ICISC, 2005, pp. 378–388.

[11] `blind-watermark` 0.4.4, https://github.com/guofei9987/blind_watermark (PyPI package).

[12] E. Koch and J. Zhao, "Towards robust and hidden image copyright labeling," in Proc. IEEE Workshop Nonlinear Signal Image Process., Neos Marmaras, Greece, Jun. 1995, pp. 452–455.

[13] S. G. Mallat, "A theory for multiresolution signal decomposition: The wavelet representation," IEEE Trans. Pattern Anal. Mach. Intell., vol. 11, no. 7, pp. 674–693, Jul. 1989.

[14] N. Ahmed, T. Natarajan, and K. R. Rao, "Discrete cosine transform," IEEE Trans. Comput., vol. C-23, no. 1, pp. 90–93, Jan. 1974, doi: 10.1109/T-C.1974.223784.

[15] G. K. Wallace, "The JPEG still picture compression standard," IEEE Trans. Consum. Electron., vol. 38, no. 1, pp. xviii–xxxiv, Feb. 1992.

[16] C.-C. Lai and C.-C. Tsai, "Digital image watermarking using discrete wavelet transform and singular value decomposition," IEEE Trans. Instrum. Meas., vol. 59, no. 11, pp. 3060–3063, Nov. 2010.

[17] C.-Y. Lin, M. Wu, J. A. Bloom, I. J. Cox, M. L. Miller, and Y. M. Lui, "Rotation, scale, and translation resilient watermarking for images," IEEE Trans. Image Process., vol. 10, no. 5, pp. 767–782, May 2001.

[18] F. Hartung and B. Girod, "Watermarking of uncompressed and compressed video," Signal Process., vol. 66, no. 3, pp. 283–301, 1998.

[19] J. Zhu, R. Kaplan, J. Johnson, and L. Fei-Fei, "HiDDeN: Hiding data with deep networks," in Proc. ECCV, 2018, pp. 657–672, doi: 10.1007/978-3-030-01267-0_40.

[20] M. Tancik, B. Mildenhall, and R. Ng, "StegaStamp: Invisible hyperlinks in physical photographs," in Proc. IEEE/CVF CVPR, 2020, pp. 2117–2126, doi: 10.1109/CVPR42600.2020.00219.

[21] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," IEEE Trans. Image Process., vol. 13, no. 4, pp. 600–612, Apr. 2004, doi: 10.1109/TIP.2003.819861.

[22] C. R. Harris et al., "Array programming with NumPy," Nature, vol. 585, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[23] P. Virtanen et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python," Nature Methods, vol. 17, no. 3, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.

[24] S. van der Walt et al., "scikit-image: Image processing in Python," PeerJ, vol. 2, p. e453, 2014, doi: 10.7717/peerj.453.

[25] G. R. Lee et al., "PyWavelets: A Python package for wavelet analysis," J. Open Source Softw., vol. 4, no. 36, p. 1237, 2019, doi: 10.21105/joss.01237.

APPENDIX A — SOURCE CODE

Python 3.13 · NumPy · SciPy · scikit-image · PyWavelets · Pillow · pandas · matplotlib (OpenCV and FFmpeg for the video test). Repository: github.com/krishangvikramsingh/claude-code, branch claude/elegant-tesla-c3k9fk, folder final_submission/source_code/src. The listing contains the algorithm, attacks, metrics, dataset builder, batch experiment, analysis and benchmark scripts; the data manifest (dataset_manifest.py), the tests and the report/presentation generators are in the repository.

A.1 watermark_core.py — Hybrid DWT–DCT–SVD embedding and extraction (algorithm core) (172 lines)

```
"""
watermark_core.py
==================
Hybrid DWT-DCT-SVD Digital Image Watermarking Engine.

Algorithm summary
------------------
EMBEDDING:
  1. Host (cover) image  -> 1-level 2D DWT (Haar)  -> subbands {LL, LH, HL, HH}
  2. Target subband (default: LL)  -> 2D DCT of the WHOLE sub-band (global, not
     block-wise; see CHANGELOG 'doc-fix-1': earlier docstrings said "block-level")
  3. DCT coefficient matrix -> SVD:            A = U . S . V^T
  4. Watermark image (grayscale, resized to subband shape) -> SVD:
                                                W = Uw . Sw . Vw^T
  5. Modify singular values of the host subband:
                                      S' = S + alpha * Sw
  6. Reconstruct:      A' = U . S' . V^T   -> inverse DCT -> modified subband
  7. Replace subband, apply inverse DWT -> watermarked image

EXTRACTION (NON-blind: requires the embedding "key" = {S_original, Uw, Vtw,
alpha, ...} saved at embedding time, and in particular the singular values of the
ORIGINAL host sub-band; this is the usual set-up for SVD ownership schemes where
the key is held by the copyright owner).  [The original text called this
"semi-blind"; because S_original is derived from the unmarked host it is
non-blind in the standard taxonomy.]
  1. Possibly-attacked watermarked image -> DWT -> target subband -> DCT -> SVD -> S_att
  2. Sw_extracted = (S_att - S_original) / alpha
  3. Watermark_extracted = Uw . Sw_extracted . Vw^T

KNOWN LIMITATION (measured in `run_batch_experiment.py`, experiment E4):
the logo's *identity* lives in the key (Uw, Vtw), not in the image.  The
extractor therefore reproduces "the key owner's logo" even when a different logo
was embedded.  A high NC value measures *fidelity of the recovered singular
values*, not proof of ownership.  This is the well-known false-positive problem
of SVD-based watermarking.

alpha (embedding strength) trades off imperceptibility (PSNR/SSIM of watermarked
image) against robustness (NC/BER of extracted watermark after attacks).

The numerical behaviour of embed_watermark / extract_watermark is IDENTICAL to
the original submission; only input validation and documentation were added.
"""

from __future__ import annotations

import numpy as np
import pywt
from scipy.fftpack import dct, idct
from skimage.transform import resize

_SUBBANDS = ("LL", "LH", "HL", "HH")


# ---------------------------------------------------------------------------
# Transform helpers
# ---------------------------------------------------------------------------

def dct2(block: np.ndarray) -> np.ndarray:
    """Orthonormal 2-D DCT-II."""
    return dct(dct(block.T, norm="ortho").T, norm="ortho")


def idct2(block: np.ndarray) -> np.ndarray:
    """Inverse of :func:`dct2`."""
    return idct(idct(block.T, norm="ortho").T, norm="ortho")


def _check_2d(name: str, a: np.ndarray, min_side: int = 8) -> np.ndarray:
    a = np.asarray(a)
    if a.ndim != 2:
        raise ValueError(f"{name} must be a 2-D greyscale array, got shape {a.shape}")
    if min(a.shape) < min_side:
        raise ValueError(f"{name} too small: {a.shape}")
```

```
    if not np.issubdtype(a.dtype, np.number):
        raise TypeError(f"{name} must be numeric, got {a.dtype}")
    if not np.isfinite(a).all():
        raise ValueError(f"{name} contains NaN/Inf")
    return a.astype(np.float64)


# ---------------------------------------------------------------------------
# Watermark embedding
# ---------------------------------------------------------------------------

def embed_watermark(host: np.ndarray, watermark: np.ndarray, alpha: float = 0.10,
                    wavelet: str = "haar", subband: str = "LL"):
    """
    Embed `watermark` (2D float array, any size) into `host` (2D float array).

    Returns
    -------
    watermarked : np.ndarray  -- same shape/dtype range as host (0-255 float)
    key : dict                -- embedding key required for extraction
    """
    host = _check_2d("host", host)
    watermark = _check_2d("watermark", watermark)
    if not alpha > 0:
        raise ValueError(f"alpha must be > 0, got {alpha}")
    if subband not in _SUBBANDS:
        raise ValueError(f"subband must be one of {_SUBBANDS}, got {subband!r}")

    # 1) DWT decomposition of host
    LL, (LH, HL, HH) = pywt.dwt2(host, wavelet)
    subbands = {"LL": LL, "LH": LH, "HL": HL, "HH": HH}
    target = subbands[subband]

    # 2) Resize watermark to match target subband size
    wm_resized = resize(watermark, target.shape, anti_aliasing=True, preserve_range=True)

    # 3) DCT of target subband
    target_dct = dct2(target)

    # 4) SVD of host (DCT domain) and watermark
    U, S, Vt = np.linalg.svd(target_dct, full_matrices=False)
    Uw, Sw, Vtw = np.linalg.svd(wm_resized, full_matrices=False)

    # 5) Modify singular values
    S_mod = S + alpha * Sw

    # 6) Reconstruct modified subband
    target_dct_mod = (U * S_mod) @ Vt
    target_mod = idct2(target_dct_mod)

    # 7) Replace subband and invert DWT
    subbands[subband] = target_mod
    watermarked = pywt.idwt2((subbands["LL"], (subbands["LH"], subbands["HL"], subbands["HH"])), wavelet)

    # Crop / pad in case IDWT introduces a 1-pixel size mismatch
    watermarked = watermarked[:host.shape[0], :host.shape[1]]
    watermarked = np.clip(watermarked, 0, 255)

    key = {
        "U": U, "Vt": Vt, "S_original": S,
        "Uw": Uw, "Vtw": Vtw,
        "alpha": alpha, "wavelet": wavelet, "subband": subband,
        "wm_shape": watermark.shape, "target_shape": target.shape,
    }
    return watermarked, key


# ---------------------------------------------------------------------------
# Watermark extraction
# ---------------------------------------------------------------------------

def extract_watermark(possibly_attacked: np.ndarray, key: dict) -> np.ndarray:
    """
    Extract the watermark from a (possibly attacked) watermarked image using
    the embedding key produced by `embed_watermark`.
    """
    img = _check_2d("image", possibly_attacked)
    wavelet = key["wavelet"]
    subband = key["subband"]

    LL, (LH, HL, HH) = pywt.dwt2(img, wavelet)
    subbands = {"LL": LL, "LH": LH, "HL": HL, "HH": HH}
    target = subbands[subband]
```

```
    # Attacks (crop / resize / rotate) can change subband size slightly;
    # force it back to the size used at embedding time.
    target_shape = key["target_shape"]
    if target.shape != target_shape:
        target = resize(target, target_shape, anti_aliasing=True, preserve_range=True)

    target_dct = dct2(target)
    _, S_att, _ = np.linalg.svd(target_dct, full_matrices=False)

    Sw_extracted = (S_att - key["S_original"]) / key["alpha"]
    watermark_extracted = (key["Uw"] * Sw_extracted) @ key["Vtw"]

    watermark_extracted = resize(watermark_extracted, key["wm_shape"],
                                 anti_aliasing=True, preserve_range=True)
    return np.clip(watermark_extracted, 0, 255)
```

A.2 metrics.py — PSNR, SSIM, MSE, NC, ZNCC, BER (102 lines)

```
"""
metrics.py
==========
Quantitative evaluation metrics for watermarking quality (imperceptibility)
and robustness (extraction fidelity under attack).

Definitions (x = reference, y = test, N = number of pixels)
    MSE  = (1/N) * sum (x - y)^2
    PSNR = 10 * log10(255^2 / MSE)                      [dB]; undefined (inf) if MSE = 0
    SSIM = mean local structural similarity (Wang et al., 2004), 7x7 uniform window
    NC   = sum(W * W') / (||W|| * ||W'||)               [0..1 for non-negative images]
    ZNCC = Pearson correlation of the two images (mean-centred NC), in [-1, 1]
    BER  = fraction of pixels whose binarised value differs

Notes
  * NC as defined here is NOT mean-centred, so two non-negative images always
    have NC >= 0 and a fully black extraction gives NC = 0 by convention.
  * PSNR is capped at 100 dB when the images are identical so that averages stay
    finite (the original code returned scikit-image's inf).
"""

import numpy as np
from skimage.metrics import peak_signal_noise_ratio as _psnr
from skimage.metrics import structural_similarity as _ssim

PSNR_CAP_DB = 100.0


def _prep(a: np.ndarray, b: np.ndarray):
    a = np.clip(np.asarray(a, dtype=np.float64), 0, 255)
    b = np.clip(np.asarray(b, dtype=np.float64), 0, 255)
    if a.shape != b.shape:
        raise ValueError(f"shape mismatch: {a.shape} vs {b.shape}")
    return a, b


def mse(original: np.ndarray, test: np.ndarray) -> float:
    a, b = _prep(original, test)
    return float(np.mean((a - b) ** 2))


def psnr(original: np.ndarray, test: np.ndarray) -> float:
    """Peak Signal-to-Noise Ratio in dB. Higher = more imperceptible."""
    a, b = _prep(original, test)
    if np.array_equal(a, b):
        return PSNR_CAP_DB
    return float(min(_psnr(a, b, data_range=255), PSNR_CAP_DB))


def ssim(original: np.ndarray, test: np.ndarray) -> float:
    """Structural Similarity Index (0-1). Higher = more structurally similar."""
    a, b = _prep(original, test)
    return float(_ssim(a, b, data_range=255))


def normalized_correlation(original_wm: np.ndarray, extracted_wm: np.ndarray) -> float:
    """
    Normalized Correlation (NC) between the original and extracted watermark.
    NC = 1.0 -> perfect extraction. NC used ubiquitously in watermarking
    literature to score robustness (typically NC > 0.75 is "recognizable").
    """
    a = np.asarray(original_wm, dtype=np.float64).flatten()
    b = np.asarray(extracted_wm, dtype=np.float64).flatten()
    if a.shape != b.shape:
```

```
        raise ValueError(f"shape mismatch: {original_wm.shape} vs {extracted_wm.shape}")
    num = np.sum(a * b)
    den = np.sqrt(np.sum(a ** 2)) * np.sqrt(np.sum(b ** 2))
    if den == 0:
        return 0.0
    return float(num / den)


def zero_normalized_correlation(original_wm: np.ndarray, extracted_wm: np.ndarray) -> float:
    """
    Mean-centred NC (Pearson correlation of the flattened images).

    Plain NC rewards any bright-ish image: a constant white image already scores
    sqrt(fill ratio) = 0.359 against the 64x64 logo, and a rank-1 approximation of the
    logo scores 0.868.  ZNCC removes the DC offset so that it measures agreement of
    the *shape* only.  Returns 0.0 if either image has zero variance.
    """
    a = np.asarray(original_wm, dtype=np.float64).flatten()
    b = np.asarray(extracted_wm, dtype=np.float64).flatten()
    if a.shape != b.shape:
        raise ValueError(f"shape mismatch: {original_wm.shape} vs {extracted_wm.shape}")
    a = a - a.mean()
    b = b - b.mean()
    den = np.sqrt(np.sum(a ** 2)) * np.sqrt(np.sum(b ** 2))
    if den == 0:
        return 0.0
    return float(np.sum(a * b) / den)


def bit_error_rate(original_wm: np.ndarray, extracted_wm: np.ndarray, thresh: float = 127.5) -> float:
    """
    BER between binarized original and extracted watermark (0-1, lower better).
    """
    a = (np.asarray(original_wm, dtype=np.float64) > thresh).astype(np.uint8).flatten()
    b = (np.asarray(extracted_wm, dtype=np.float64) > thresh).astype(np.uint8).flatten()
    if a.shape != b.shape:
        raise ValueError(f"shape mismatch: {original_wm.shape} vs {extracted_wm.shape}")
    return float(np.mean(a != b))
```

A.3 attacks.py — The 14 attacks plus the unattacked reference, seeded (141 lines)

```
"""
attacks.py
==========
Standard attack/distortion suite used to benchmark watermark robustness.
Each function takes a float image (0-255) and returns a same-size float image.

Changes w.r.t. the original submission (see CHANGELOG):
  * the two stochastic attacks now accept a ``rng`` seed -> runs are reproducible
  * ``make_attack_suite(seed)`` builds the 15-condition suite (14 attacks + "No Attack") with a given seed;
    ``ATTACK_SUITE`` (seed 0) is kept for backward compatibility
  * documentation of what "Cropping 25%" really does (see below)
  * JPEG attack validates its input and always returns a 2-D array
"""

from __future__ import annotations

import io

import numpy as np
from PIL import Image
from scipy import ndimage
from skimage.exposure import equalize_hist
from skimage.transform import rotate, resize
from skimage.util import random_noise


def jpeg_compression(img: np.ndarray, quality: int = 50) -> np.ndarray:
    """Round-trip through Pillow's baseline JPEG encoder at the given quality (1-95)."""
    if not 1 <= quality <= 95:
        raise ValueError("quality must be in 1..95")
    im = Image.fromarray(np.clip(np.rint(img), 0, 255).astype(np.uint8))
    buf = io.BytesIO()
    im.save(buf, format="JPEG", quality=quality)
    buf.seek(0)
    return np.array(Image.open(buf).convert("L")).astype(np.float64)


def gaussian_noise(img: np.ndarray, var: float = 0.01, rng=None) -> np.ndarray:
    """Additive Gaussian noise; ``var`` is the variance on the [0,1] intensity scale."""
    noisy = random_noise(img / 255.0, mode="gaussian", var=var, rng=rng)
    return noisy * 255.0
```

```
def salt_pepper_noise(img: np.ndarray, amount: float = 0.02, rng=None) -> np.ndarray:
    """Replace ``amount`` of the pixels by 0 or 255 with equal probability."""
    noisy = random_noise(img / 255.0, mode="s&p", amount=amount, rng=rng)
    return noisy * 255.0


def median_filter(img: np.ndarray, size: int = 3) -> np.ndarray:
    return ndimage.median_filter(img, size=size)


def gaussian_blur(img: np.ndarray, sigma: float = 1.5) -> np.ndarray:
    return ndimage.gaussian_filter(img, sigma=sigma)


def rotation_attack(img: np.ndarray, angle: float = 5.0) -> np.ndarray:
    """Rotate about the centre (counter-clockwise, degrees), keep size, replicate edges.
    No re-alignment is applied before extraction (a pure geometric-attack test)."""
    rotated = rotate(img / 255.0, angle, resize=False, mode="edge")
    return rotated * 255.0


def cropping_attack(img: np.ndarray, frac: float = 0.25) -> np.ndarray:
    """Zero a border so that ``frac`` of EACH DIMENSION is removed (frac/2 per side).

    For frac = 0.25 on a 512x512 image this blacks out a 64-px frame on every
    side, i.e. the surviving centre is 384x384 = 56.25 % of the area, so
    43.75 % of the *area* is removed.  (The original report called this
    "Cropping 25%"; the label is kept for comparability but the report now states
    the area fraction explicitly.)  The image is not resized, so geometry is kept.
    """
    h, w = img.shape
    ch, cw = int(h * frac / 2), int(w * frac / 2)
    out = img.copy()
    out[:ch, :] = 0
    out[h - ch:, :] = 0
    out[:, :cw] = 0
    out[:, w - cw:] = 0
    return out


def scaling_attack(img: np.ndarray, factor: float = 0.5) -> np.ndarray:
    h, w = img.shape
    small = resize(img, (int(h * factor), int(w * factor)), anti_aliasing=True, preserve_range=True)
    back = resize(small, (h, w), anti_aliasing=True, preserve_range=True)
    return back


def histogram_equalization(img: np.ndarray) -> np.ndarray:
    eq = equalize_hist(img / 255.0)
    return eq * 255.0


def sharpening_attack(img: np.ndarray, amount: float = 1.0) -> np.ndarray:
    blurred = ndimage.gaussian_filter(img, sigma=1.0)
    sharpened = img + amount * (img - blurred)
    return np.clip(sharpened, 0, 255)


ATTACK_NAMES = [
    "No Attack", "JPEG Q=90", "JPEG Q=50", "JPEG Q=20",
    "Gaussian Noise (var=0.005)", "Gaussian Noise (var=0.02)", "Salt & Pepper (2%)",
    "Median Filter 3x3", "Gaussian Blur (sigma=1.5)", "Rotation 5 deg", "Rotation 15 deg",
    "Cropping 25%", "Scaling 0.5x", "Histogram Equalization", "Sharpening",
]

ATTACK_FAMILY = {
    "No Attack": "none", "JPEG Q=90": "compression", "JPEG Q=50": "compression", "JPEG Q=20": "compression",
    "Gaussian Noise (var=0.005)": "noise", "Gaussian Noise (var=0.02)": "noise", "Salt & Pepper (2%)": "noise",
    "Median Filter 3x3": "filtering", "Gaussian Blur (sigma=1.5)": "filtering", "Sharpening": "filtering",
    "Rotation 5 deg": "geometric", "Rotation 15 deg": "geometric", "Cropping 25%": "geometric",
    "Scaling 0.5x": "geometric", "Histogram Equalization": "global intensity",
}


def make_attack_suite(seed: int = 0) -> dict:
    """Return {name: callable} for the 15-condition suite (14 attacks + "No Attack"); stochastic attacks use
``seed``."""
    return {
        "No Attack":                 lambda x: x.copy(),
        "JPEG Q=90":                 lambda x: jpeg_compression(x, 90),
        "JPEG Q=50":                 lambda x: jpeg_compression(x, 50),
        "JPEG Q=20":                 lambda x: jpeg_compression(x, 20),
```

```
        "Gaussian Noise (var=0.005)": lambda x: gaussian_noise(x, 0.005, rng=seed),
        "Gaussian Noise (var=0.02)":  lambda x: gaussian_noise(x, 0.02, rng=seed + 1),
        "Salt & Pepper (2%)":         lambda x: salt_pepper_noise(x, 0.02, rng=seed + 2),
        "Median Filter 3x3":          lambda x: median_filter(x, 3),
        "Gaussian Blur (sigma=1.5)":  lambda x: gaussian_blur(x, 1.5),
        "Rotation 5 deg":             lambda x: rotation_attack(x, 5),
        "Rotation 15 deg":            lambda x: rotation_attack(x, 15),
        "Cropping 25%":               lambda x: cropping_attack(x, 0.25),
        "Scaling 0.5x":               lambda x: scaling_attack(x, 0.5),
        "Histogram Equalization":     lambda x: histogram_equalization(x),
        "Sharpening":                 lambda x: sharpening_attack(x, 1.0),
    }


# Backward-compatible module-level suite (seed 0).
ATTACK_SUITE = make_attack_suite(0)
assert list(ATTACK_SUITE) == ATTACK_NAMES
```

A.4 logo.py — Procedural 64×64 binary watermark logo (25 lines)

```
"""
logo.py
=======
Procedural binary watermark logo (moved out of run_experiment.py unchanged so that
every script uses exactly the same watermark; no external/copyrighted asset).
"""

import numpy as np
from PIL import Image, ImageDraw, ImageFont


def make_watermark_logo(size=64, text="CYBERION"):
    """Procedurally generate a binary logo watermark (no external/copyrighted asset)."""
    img = Image.new("L", (size * 4, size), color=0)
    draw = ImageDraw.Draw(img)
    try:
        font = ImageFont.load_default()
    except Exception:
        font = None
    draw.rectangle([0, 0, size * 4 - 1, size - 1], outline=255, width=3)
    draw.text((10, size // 2 - 6), text, fill=255, font=font)
    img = img.resize((size, size))
    arr = np.array(img).astype(np.float64)
    arr = (arr > 100).astype(np.float64) * 255.0
    return arr
```

A.5 image_io.py — Safe image loading and standardisation (111 lines)

```
"""
image_io.py
===========
Safe image loading and standardisation for the watermarking experiments.

The original project only ever handled two scikit-image samples that were
already single-channel arrays.  A heterogeneous 100-image set needs explicit
handling of colour modes, alpha channels, 16-bit data, odd aspect ratios and
corrupted files, otherwise results silently depend on how Pillow happens to
decode each file.

Standardisation rule (documented in the report, Section 5.1)
------------------------------------------------------------
1. Decode with Pillow; reject empty / corrupt / unsupported files.
2. Alpha channels are composited over white.
3. 16-bit / float data is linearly rescaled to 8 bit.
4. Colour -> grey with Pillow's ITU-R BT.601 luma  L = 0.299 R + 0.587 G + 0.114 B.
5. Centre-crop to a square (largest possible) so aspect ratio is not distorted.
6. Resize to ``size`` x ``size`` with Lanczos resampling.
"""

from __future__ import annotations

import os
from dataclasses import dataclass

import numpy as np
from PIL import Image, UnidentifiedImageError

Image.MAX_IMAGE_PIXELS = 200_000_000  # allow big panoramas, still bounded


class ImageLoadError(RuntimeError):
    """Raised when an image cannot be decoded or is unusable."""
```

```
@dataclass
class SourceInfo:
    width: int
    height: int
    mode: str
    fmt: str
    file_size: int


def _to_uint8_rgbish(im: Image.Image) -> Image.Image:
    """Return an 8-bit 'L' or 'RGB' image regardless of the source mode."""
    mode = im.mode
    if mode in ("RGBA", "LA", "PA") or (mode == "P" and "transparency" in im.info):
        im = im.convert("RGBA")
        bg = Image.new("RGBA", im.size, (255, 255, 255, 255))
        return Image.alpha_composite(bg, im).convert("RGB")
    if mode in ("I;16", "I;16B", "I;16L", "I", "F"):
        arr = np.asarray(im, dtype=np.float64)
        lo, hi = float(arr.min()), float(arr.max())
        if hi <= lo:
            raise ImageLoadError("constant-valued high-bit-depth image")
        return Image.fromarray(((arr - lo) / (hi - lo) * 255.0).round().astype(np.uint8), "L")
    if mode == "1":
        return im.convert("L")
    if mode in ("L", "RGB"):
        return im
    return im.convert("RGB")


def load_gray_square(path: str, size: int = 512) -> tuple[np.ndarray, SourceInfo]:
    """Load ``path`` and return a ``size`` x ``size`` float64 array in [0, 255]."""
    if not os.path.isfile(path):
        raise ImageLoadError(f"file not found: {path}")
    fsize = os.path.getsize(path)
    if fsize == 0:
        raise ImageLoadError(f"empty file: {path}")
    try:
        with Image.open(path) as im:
            fmt = im.format or ""
            w, h = im.size
            mode = im.mode
            im.load()
            rgb = _to_uint8_rgbish(im)
    except (UnidentifiedImageError, OSError, ValueError, SyntaxError) as exc:
        raise ImageLoadError(f"cannot decode {os.path.basename(path)}: {exc}") from exc
    if min(w, h) < 16:
        raise ImageLoadError(f"image too small ({w}x{h})")

    gray = rgb.convert("L")
    side = min(gray.size)
    left, top = (gray.width - side) // 2, (gray.height - side) // 2
    gray = gray.crop((left, top, left + side, top + side)).resize((size, size), Image.LANCZOS)
    arr = np.asarray(gray, dtype=np.float64)
    return arr, SourceInfo(w, h, mode, fmt, fsize)


def save_gray_png(arr: np.ndarray, path: str) -> None:
    """Write an 8-bit greyscale PNG (values rounded and clipped)."""
    os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
    Image.fromarray(np.clip(np.rint(arr), 0, 255).astype(np.uint8), "L").save(path)


def verify_png(path: str, expected_shape: tuple[int, int] | None = None) -> bool:
    """Re-open ``path`` and check it decodes to a non-degenerate 2-D uint8 image."""
    try:
        with Image.open(path) as im:
            im.load()
            a = np.asarray(im)
    except Exception:
        return False
    if a.ndim != 2 or a.dtype != np.uint8:
        return False
    if expected_shape is not None and a.shape != tuple(expected_shape):
        return False
    return bool(np.isfinite(a).all())
```

A.6 run_experiment.py — Original two-image experiment (reproduces the first report) (175 lines)

```
"""
run_experiment.py
=================
LEGACY 2-IMAGE EXPERIMENT (reproduces the numbers in the submitted report).
```

```
The 100-image evaluation lives in run_batch_experiment.py.  Differences to the
original file: the logo generator moved to logo.py (identical output), results go
to results/legacy_two_image/, stochastic attacks are now seeded.

End-to-end pipeline:
  1. Load host (cover) images -- standard grayscale test images
  2. Generate a synthetic binary watermark logo (procedurally drawn, not
     copyrighted material)
  3. Embed watermark using the hybrid DWT-DCT-SVD scheme (watermark_core.py)
  4. Measure imperceptibility (PSNR / SSIM) of the watermarked image
  5. Run the full attack suite (attacks.py)
  6. Extract watermark after each attack, measure NC / BER (metrics.py)
  7. Save all figures + a results CSV/JSON to ../results/
"""

import os
import json
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from PIL import Image
from skimage import data as skdata
from skimage.transform import resize

from watermark_core import embed_watermark, extract_watermark
from metrics import psnr, ssim, normalized_correlation, bit_error_rate
from attacks import ATTACK_SUITE
from logo import make_watermark_logo  # noqa: F401  (re-exported for backward compatibility)

RESULTS_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "..", "..", "results",
"legacy_two_image")
os.makedirs(RESULTS_DIR, exist_ok=True)


def load_host_images():
    hosts = {
        "cameraman": resize(skdata.camera().astype(np.float64), (512, 512), preserve_range=True),
        "astronaut_gray": resize(skdata.astronaut().mean(axis=2).astype(np.float64), (512, 512),
preserve_range=True),
    }
    return hosts


def save_image(arr, path):
    Image.fromarray(np.clip(arr, 0, 255).astype(np.uint8)).save(path)


def main():
    alpha = 0.10
    hosts = load_host_images()
    watermark = make_watermark_logo(size=64, text="CYBERION")
    save_image(watermark, os.path.join(RESULTS_DIR, "watermark_logo.png"))

    all_rows = []
    per_host_extracted = {}

    for host_name, host in hosts.items():
        watermarked, key = embed_watermark(host, watermark, alpha=alpha)
        save_image(host, os.path.join(RESULTS_DIR, f"{host_name}_original.png"))
        save_image(watermarked, os.path.join(RESULTS_DIR, f"{host_name}_watermarked.png"))

        imperceptibility_psnr = psnr(host, watermarked)
        imperceptibility_ssim = ssim(host, watermarked)

        extracted_examples = {}
        for attack_name, attack_fn in ATTACK_SUITE.items():
            attacked = attack_fn(watermarked)
            extracted = extract_watermark(attacked, key)
            nc = normalized_correlation(watermark, extracted)
            ber = bit_error_rate(watermark, extracted)

            all_rows.append({
                "Host Image": host_name,
                "Attack": attack_name,
                "Watermarked PSNR (dB)": round(imperceptibility_psnr, 3),
                "Watermarked SSIM": round(imperceptibility_ssim, 4),
                "Extracted NC": round(nc, 4),
                "Extracted BER": round(ber, 4),
            })
            extracted_examples[attack_name] = extracted
```

```
        per_host_extracted[host_name] = extracted_examples

    df = pd.DataFrame(all_rows)
    df.to_csv(os.path.join(RESULTS_DIR, "results_table.csv"), index=False)
    print(df.to_string(index=False))

    # ---- Figure 1: original / watermarked / difference (per host) ----
    for host_name, host in hosts.items():
        watermarked = np.array(Image.open(os.path.join(RESULTS_DIR,
f"{host_name}_watermarked.png"))).astype(np.float64)
        diff = np.abs(host - watermarked)
        fig, axes = plt.subplots(1, 3, figsize=(12, 4.2))
        axes[0].imshow(host, cmap="gray"); axes[0].set_title("Original Host"); axes[0].axis("off")
        axes[1].imshow(watermarked, cmap="gray"); axes[1].set_title("Watermarked"); axes[1].axis("off")
        im = axes[2].imshow(diff, cmap="hot"); axes[2].set_title("Absolute Difference (x1)");
axes[2].axis("off")
        fig.colorbar(im, ax=axes[2], fraction=0.046)
        plt.tight_layout()
        plt.savefig(os.path.join(RESULTS_DIR, f"{host_name}_comparison.png"), dpi=150)
        plt.close(fig)

    # ---- Figure 2: watermark extraction gallery under attacks (cameraman) ----
    gallery_attacks = ["No Attack", "JPEG Q=50", "JPEG Q=20", "Gaussian Noise (var=0.02)",
                       "Median Filter 3x3", "Rotation 15 deg", "Cropping 25%", "Scaling 0.5x"]
    fig, axes = plt.subplots(2, 4, figsize=(14, 7))
    for ax, aname in zip(axes.flat, gallery_attacks):
        ex = per_host_extracted["cameraman"][aname]
        ax.imshow(ex, cmap="gray")
        row = df[(df["Host Image"] == "cameraman") & (df["Attack"] == aname)].iloc[0]
        ax.set_title(f"{aname}\nNC={row['Extracted NC']:.3f}", fontsize=9)
        ax.axis("off")
    plt.tight_layout()
    plt.savefig(os.path.join(RESULTS_DIR, "extraction_gallery.png"), dpi=150)
    plt.close(fig)

    # ---- Figure 3: NC bar chart across attacks ----
    sub = df[df["Host Image"] == "cameraman"].sort_values("Extracted NC")
    fig, ax = plt.subplots(figsize=(9, 6))
    colors = ["#c0392b" if v < 0.75 else "#27ae60" for v in sub["Extracted NC"]]
    ax.barh(sub["Attack"], sub["Extracted NC"], color=colors)
    ax.axvline(0.75, color="black", linestyle="--", linewidth=1, label="Recognizability threshold (NC=0.75)")
    ax.set_xlabel("Normalized Correlation (NC)")
    ax.set_title("Watermark Robustness Across Attack Types (host: cameraman)")
    ax.legend(loc="lower right")
    plt.tight_layout()
    plt.savefig(os.path.join(RESULTS_DIR, "nc_bar_chart.png"), dpi=150)
    plt.close(fig)

    # ---- Figure 4: alpha sensitivity sweep (PSNR vs NC tradeoff) ----
    host = hosts["cameraman"]
    alphas = [0.02, 0.05, 0.08, 0.10, 0.15, 0.20, 0.30, 0.40]
    sweep_rows = []
    for a in alphas:
        wm_img, k = embed_watermark(host, watermark, alpha=a)
        p = psnr(host, wm_img)
        attacked = ATTACK_SUITE["JPEG Q=50"](wm_img)
        ext = extract_watermark(attacked, k)
        nc = normalized_correlation(watermark, ext)
        sweep_rows.append({"alpha": a, "PSNR": p, "NC_after_JPEG50": nc})
    sweep_df = pd.DataFrame(sweep_rows)
    sweep_df.to_csv(os.path.join(RESULTS_DIR, "alpha_sweep.csv"), index=False)

    fig, ax1 = plt.subplots(figsize=(8, 5.5))
    ax2 = ax1.twinx()
    ax1.plot(sweep_df["alpha"], sweep_df["PSNR"], "o-", color="#2980b9", label="PSNR (imperceptibility)")
    ax2.plot(sweep_df["alpha"], sweep_df["NC_after_JPEG50"], "s-", color="#e67e22", label="NC after JPEG Q=50
(robustness)")
    ax1.set_xlabel("Embedding strength (alpha)")
    ax1.set_ylabel("PSNR (dB)", color="#2980b9")
    ax2.set_ylabel("NC after JPEG-50 attack", color="#e67e22")
    ax1.set_title("Imperceptibility vs. Robustness Trade-off")
    fig.legend(loc="upper center", bbox_to_anchor=(0.5, 0.02), ncol=2)
    plt.tight_layout()
    plt.savefig(os.path.join(RESULTS_DIR, "alpha_tradeoff.png"), dpi=150, bbox_inches="tight")
    plt.close(fig)

    summary = {
        "alpha_used_main_experiment": alpha,
        "hosts": list(hosts.keys()),
        "num_attacks_tested": len(ATTACK_SUITE),
        "mean_watermarked_psnr_db": round(df["Watermarked PSNR (dB)"].mean(), 3),
        "mean_watermarked_ssim": round(df["Watermarked SSIM"].mean(), 4),
        "mean_nc_all_attacks": round(df["Extracted NC"].mean(), 4),
```

```
        "attacks_with_NC_above_0.75": int((df["Extracted NC"] > 0.75).sum()),
        "attacks_total_rows": len(df),
    }
    with open(os.path.join(RESULTS_DIR, "summary.json"), "w") as f:
        json.dump(summary, f, indent=2)
    print("\nSUMMARY:", json.dumps(summary, indent=2))


if __name__ == "__main__":
    main()
```

A.7 build_dataset.py — Reproducible 100-image dataset builder (244 lines)

```
"""
build_dataset.py
================
Reproducibly assemble the 100-image evaluation set.

    python build_dataset.py [--out ../../data] [--target 100]

Steps
-----
1. Download every entry of ``dataset_manifest.REAL_IMAGES`` (cached in
   ``<out>/source_images``; SHA-256 recorded).  Unreachable sources are logged
   and skipped - nothing is invented to replace them.
2. Standardise (see ``image_io``) to 512x512 8-bit greyscale PNG.
3. Reject near-duplicates (64-bit difference hash, Hamming distance <= 6).
4. Top the set up to ``--target`` images with clearly *labelled synthetic*
   control images (fixed seeds).  Synthetic images are never presented as
   real-world data; they form a separate stratum in every table.
5. Write ``original/image_XXX.png``, ``metadata.csv`` and a build log.
"""

from __future__ import annotations

import argparse
import csv
import hashlib
import json
import logging
import os
import sys
import time
import urllib.parse
import urllib.request

import numpy as np
from PIL import Image
from scipy import ndimage

from dataset_manifest import REAL_IMAGES
from image_io import ImageLoadError, load_gray_square, save_gray_png

LOG = logging.getLogger("build_dataset")
RAW = "https://raw.githubusercontent.com/"
SIZE = 512


# ---------------------------------------------------------------------------
# helpers
# ---------------------------------------------------------------------------

def sha256_file(path: str) -> str:
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(1 << 20), b""):
            h.update(chunk)
    return h.hexdigest()


def dhash64(arr: np.ndarray) -> int:
    """64-bit difference hash of a greyscale array."""
    im = Image.fromarray(np.clip(arr, 0, 255).astype(np.uint8)).resize((9, 8), Image.LANCZOS)
    a = np.asarray(im, dtype=np.int16)
    bits = (a[:, 1:] > a[:, :-1]).flatten()
    return int("".join("1" if b else "0" for b in bits), 2)


def hamming(a: int, b: int) -> int:
    return bin(a ^ b).count("1")


def download(rel: str, dest: str, retries: int = 3) -> bool:
```

```
    url = RAW + rel
    last = None
    for k in range(retries):
        try:
            with urllib.request.urlopen(urllib.request.Request(url, headers={"User-Agent": "ivp-watermark-
build/1.0"}), timeout=60) as r:
                data = r.read()
            if not data:
                raise IOError("empty response")
            with open(dest, "wb") as f:
                f.write(data)
            return True
        except Exception as exc:  # noqa: BLE001 - log and retry
            last = exc
            time.sleep(1.5 * (k + 1))
    LOG.warning("download failed %s (%s)", url, last)
    return False


def fetch_scipy(name: str, dest: str) -> bool:
    try:
        import scipy.datasets as sd
        arr = getattr(sd, name)()
        Image.fromarray(arr).save(dest)
        return True
    except Exception as exc:  # noqa: BLE001
        LOG.warning("scipy.datasets.%s unavailable: %s", name, exc)
        return False


# ---------------------------------------------------------------------------
# synthetic control images (all deterministic, labelled as synthetic)
# ---------------------------------------------------------------------------

def _smooth_noise(rng, sigma, mean, std):
    n = ndimage.gaussian_filter(rng.standard_normal((SIZE, SIZE)), sigma)
    n = (n - n.mean()) / n.std()
    return np.clip(mean + std * n, 0, 255)


def synth_images():
    rng = np.random.default_rng(20260713)
    yy, xx = np.mgrid[0:SIZE, 0:SIZE].astype(np.float64)
    out = []
    out.append(("syn_hgradient", "horizontal linear gradient 0-255", np.tile(np.linspace(0, 255, SIZE), (SIZE,
1))))
    r = np.hypot(xx - SIZE / 2, yy - SIZE / 2)
    out.append(("syn_radial", "radial gradient (smooth, low detail)", np.clip(255 - r * 0.7, 0, 255)))
    out.append(("syn_grating", "2-D sinusoidal grating (periodic texture)", 128 + 100 * np.sin(2 * np.pi * (xx
* 9 + yy * 5) / SIZE)))
    out.append(("syn_white_noise", "white Gaussian noise, mean 128 / std 40", np.clip(128 + 40 *
rng.standard_normal((SIZE, SIZE)), 0, 255)))
    out.append(("syn_lowcontrast", "smooth noise, mean 128 / std 8 (very low contrast)", _smooth_noise(rng, 6,
128, 8)))
    out.append(("syn_checker", "32-px checkerboard (hard edges)", 255.0 * (((xx // 32) + (yy // 32)) % 2)))
    out.append(("syn_dark", "smooth noise, mean 25 / std 12 (dark)", _smooth_noise(rng, 5, 25, 12)))
    out.append(("syn_bright", "smooth noise, mean 230 / std 12 (bright)", _smooth_noise(rng, 5, 230, 12)))
    # 1/f-like fractal texture
    f = np.fft.fftfreq(SIZE)
    fx, fy = np.meshgrid(f, f)
    amp = 1.0 / np.maximum(np.hypot(fx, fy), 1.0 / SIZE)
    spec = amp * np.exp(2j * np.pi * rng.random((SIZE, SIZE)))
    frac = np.real(np.fft.ifft2(spec))
    frac = (frac - frac.mean()) / frac.std()
    out.append(("syn_fractal", "1/f fractal noise texture", np.clip(128 + 45 * frac, 0, 255)))
    # random blocks scene
    scene = np.full((SIZE, SIZE), 110.0)
    for _ in range(14):
        x0, y0 = rng.integers(0, SIZE - 80, 2)
        w, h = rng.integers(40, 160, 2)
        scene[y0:y0 + h, x0:x0 + w] = rng.integers(20, 240)
    out.append(("syn_blocks", "random grey rectangles on grey background", scene))
    return out


# ---------------------------------------------------------------------------
# main
# ---------------------------------------------------------------------------

def main():
    ap = argparse.ArgumentParser()
    here = os.path.dirname(os.path.abspath(__file__))
    ap.add_argument("--out", default=os.path.join(here, "..", "..", "data"))
```

```
    ap.add_argument("--target", type=int, default=100)
    ap.add_argument("--dup-threshold", type=int, default=6)
    args = ap.parse_args()
    out = os.path.abspath(args.out)
    src_dir = os.path.join(out, "source_images")
    orig_dir = os.path.join(out, "original")
    os.makedirs(src_dir, exist_ok=True)
    os.makedirs(orig_dir, exist_ok=True)
    logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")

    rows, hashes, skipped = [], [], []
    for key, rel, group, lic, note, category in REAL_IMAGES:
        ext = os.path.splitext(urllib.parse.unquote(rel))[1] or ".png"
        if rel.startswith("scipy:"):
            dest = os.path.join(src_dir, f"{key}.png")
            ok = os.path.isfile(dest) or fetch_scipy(rel.split(":", 1)[1], dest)
            url = f"scipy.datasets.{rel.split(':', 1)[1]}()"
        else:
            dest = os.path.join(src_dir, f"{key}{ext}")
            ok = os.path.isfile(dest) and os.path.getsize(dest) > 0 or download(rel, dest)
            url = RAW + rel
        if not ok:
            skipped.append((key, "download failed"))
            continue
        try:
            arr, info = load_gray_square(dest, SIZE)
        except ImageLoadError as exc:
            skipped.append((key, str(exc)))
            LOG.warning("skip %s: %s", key, exc)
            continue
        if min(info.width, info.height) < 240:
            skipped.append((key, f"too small ({info.width}x{info.height})"))
            continue
        hsh = dhash64(arr)
        dup = next((k for k, h in hashes if hamming(h, hsh) <= args.dup_threshold), None)
        if dup:
            skipped.append((key, f"near-duplicate of {dup}"))
            LOG.info("duplicate %s ~ %s", key, dup)
            continue
        hashes.append((key, hsh))
        rows.append(dict(key=key, arr=arr, url=url, group=group, lic=lic, note=note,
                         category=category, info=info, sha=sha256_file(dest),
                         fname=os.path.basename(dest), synthetic=False))

    need = args.target - len(rows)
    if need < 0:
        rows = rows[:args.target]
        need = 0
    synth = synth_images()
    if need > len(synth):
        LOG.warning("only %d synthetic generators available; dataset will have %d images",
                    len(synth), len(rows) + len(synth))
    for key, desc, arr in synth[:need]:
        rows.append(dict(key=key, arr=arr, url="generated by build_dataset.py (seeded)", group="synthetic
control",
                         lic="n/a (generated)", note=desc, category="synthetic_control",
                         info=None, sha="", fname="", synthetic=True))

    meta_path = os.path.join(out, "metadata.csv")
    cols = ["image_id", "key", "original_filename", "source_group", "source_url", "repository_license",
            "rights_note", "original_width", "original_height", "original_mode", "original_format",
            "original_file_size_bytes", "source_sha256", "standardized_shape", "standardized_channels",
            "category", "is_synthetic", "mean_intensity", "contrast_std", "brightness_class",
            "contrast_class", "applicable_operations"]
    with open(meta_path, "w", newline="", encoding="utf-8") as f:
        w = csv.DictWriter(f, fieldnames=cols)
        w.writeheader()
        for i, r in enumerate(rows, 1):
            iid = f"image_{i:03d}"
            save_gray_png(r["arr"], os.path.join(orig_dir, iid + ".png"))
            a = np.rint(r["arr"])
            mean, std = float(a.mean()), float(a.std())
            info = r["info"]
            w.writerow(dict(
                image_id=iid, key=r["key"], original_filename=r["fname"], source_group=r["group"],
                source_url=r["url"], repository_license=r["lic"], rights_note=r["note"],
                original_width=info.width if info else SIZE, original_height=info.height if info else SIZE,
                original_mode=info.mode if info else "generated", original_format=info.fmt if info else "PNG",
                original_file_size_bytes=info.file_size if info else "", source_sha256=r["sha"],
                standardized_shape=f"{SIZE}x{SIZE}", standardized_channels=1, category=r["category"],
                is_synthetic=int(r["synthetic"]), mean_intensity=round(mean, 2), contrast_std=round(std, 2),
                brightness_class="dark" if mean < 80 else ("bright" if mean > 170 else "mid"),
                contrast_class="low" if std < 35 else ("high" if std > 65 else "medium"),
```

```
                applicable_operations="embed;extract;14 attacks + reference;PSNR;SSIM;NC;BER;alpha-
sweep;controls"))

    with open(os.path.join(out, "build_log.json"), "w") as f:
        json.dump({"images_written": len(rows), "real": sum(not r["synthetic"] for r in rows),
                   "synthetic": sum(r["synthetic"] for r in rows), "skipped": skipped}, f, indent=2)
    LOG.info("wrote %d images (%d real, %d synthetic); skipped %d",
             len(rows), sum(not r["synthetic"] for r in rows), sum(r["synthetic"] for r in rows), len(skipped))
    for k, why in skipped:
        LOG.info("  skipped %-16s %s", k, why)
    return 0 if len(rows) == args.target else 1


if __name__ == "__main__":
    sys.exit(main())
```

A.8 run_batch_experiment.py — 100-image batch experiment (E1–E4) (301 lines)

```
"""
run_batch_experiment.py
=======================
Batch evaluation of the hybrid DWT-DCT-SVD watermarking scheme on the
100-image dataset (``data/original``).

    python run_batch_experiment.py [--config config.json] [--workers 4]

Experiments
-----------
E1  imperceptibility  : PSNR / SSIM of every watermarked image (alpha = config.alpha)
E2  robustness        : 15 conditions (14 attacks + an unattacked reference) x 2 protocols ("uint8":
watermarked image is first
                        rounded to 8 bit as it would be when saved to disk; "float":
                        the original submission's protocol, attacks act on floats)
E3  alpha sweep       : PSNR / NC / ZNCC / BER vs alpha for JPEG Q=50, Gaussian noise,
                        Gaussian blur and 5-degree rotation
E4  false-positive controls (see watermark_core docstring):
      a) unwatermarked *other* host, key of image i
      b) watermarked *other* host (logo A embedded), key of image i
      c) same host watermarked with a DIFFERENT logo, decoded with logo-A key
      d) same host, unwatermarked, own key (degenerate by construction)

Every operation is wrapped so that a failure is recorded (status / error text)
instead of aborting the run; outputs are re-opened and validated.
"""

from __future__ import annotations

import os

# one BLAS thread per worker: reproducible timings, no oversubscription
for _v in ("OPENBLAS_NUM_THREADS", "OMP_NUM_THREADS", "MKL_NUM_THREADS"):
    os.environ.setdefault(_v, "1")

import argparse
import concurrent.futures as cf
import json
import logging
import platform
import resource
import sys
import time

import numpy as np
import pandas as pd

from attacks import ATTACK_FAMILY, ATTACK_NAMES, make_attack_suite
from image_io import save_gray_png, verify_png
from logo import make_watermark_logo
from metrics import bit_error_rate, normalized_correlation, psnr, ssim, zero_normalized_correlation
from watermark_core import embed_watermark, extract_watermark

HERE = os.path.dirname(os.path.abspath(__file__))
LOG = logging.getLogger("batch")

DEFAULT_CONFIG = {
    "alpha": 0.10,
    "image_size": 512,
    "logo_size": 64,
    "logo_text": "CYBERION",
    "other_logos": ["OTHERLOGO", "OWNER-B"],
    "protocols": ["uint8", "float"],
    "primary_protocol": "uint8",
    "base_seed": 20260713,
```

```
    "alphas": [0.02, 0.05, 0.08, 0.10, 0.15, 0.20, 0.30, 0.40],
    "sweep_attacks": ["JPEG Q=50", "Gaussian Noise (var=0.02)", "Gaussian Blur (sigma=1.5)", "Rotation 5 deg"],
    "n_controls_per_image": 5,
    "nc_threshold": 0.75,
    "data_dir": "../../data",
    "results_dir": "../../results",
    "outputs_dir": "../../outputs",
}


def _t():
    return time.perf_counter()


def _load_host(data_dir, image_id):
    from PIL import Image
    a = np.asarray(Image.open(os.path.join(data_dir, "original", image_id + ".png")), dtype=np.float64)
    if a.ndim != 2:
        raise ValueError(f"{image_id}: expected greyscale, got shape {a.shape}")
    return a


def _quantize(a):
    return np.clip(np.rint(a), 0, 255)


# ---------------------------------------------------------------------------
# per-image worker
# ---------------------------------------------------------------------------

def process_image(task):
    """Run E1-E4 for one image. Returns plain Python data (picklable)."""
    idx, image_id, cfg, others = task
    data_dir = cfg["data_dir"]
    rows_att, rows_ctl, rows_sw = [], [], []
    rec = {"image_id": image_id, "status": "ok", "error": ""}
    try:
        host = _load_host(data_dir, image_id)
        logo = make_watermark_logo(cfg["logo_size"], cfg["logo_text"])
        alpha = cfg["alpha"]

        # ---- E1: embedding ------------------------------------------------
        t0 = _t()
        wm_float, key = embed_watermark(host, logo, alpha=alpha)
        rec["t_embed_ms"] = (_t() - t0) * 1e3
        wm_u8 = _quantize(wm_float)
        out_path = os.path.join(cfg["outputs_dir"], "watermarked", f"{image_id}_wm.png")
        save_gray_png(wm_u8, out_path)
        rec["output_valid"] = bool(verify_png(out_path, host.shape))
        rec["psnr_float_db"] = psnr(host, wm_float)
        rec["ssim_float"] = ssim(host, wm_float)
        rec["psnr_db"] = psnr(host, wm_u8)          # primary: image as stored on disk
        rec["ssim"] = ssim(host, wm_u8)
        rec["max_abs_diff"] = float(np.abs(host - wm_u8).max())
        rec["mean_abs_diff"] = float(np.abs(host - wm_u8).mean())
        rec["key_bytes"] = int(sum(v.nbytes for v in key.values() if isinstance(v, np.ndarray)))

        # ---- E2: attacks --------------------------------------------------
        suite = make_attack_suite(cfg["base_seed"] + 100 * idx)
        for protocol in cfg["protocols"]:
            base = wm_u8 if protocol == "uint8" else wm_float
            for name in ATTACK_NAMES:
                row = {"image_id": image_id, "protocol": protocol, "attack": name,
                       "family": ATTACK_FAMILY[name], "status": "ok", "error": ""}
                try:
                    t0 = _t()
                    attacked = suite[name](base)
                    row["t_attack_ms"] = (_t() - t0) * 1e3
                    t0 = _t()
                    ext = extract_watermark(attacked, key)
                    row["t_extract_ms"] = (_t() - t0) * 1e3
                    row["nc"] = normalized_correlation(logo, ext)
                    row["zncc"] = zero_normalized_correlation(logo, ext)
                    row["ber"] = bit_error_rate(logo, ext)
                except Exception as exc:  # noqa: BLE001
                    row.update(status="error", error=f"{type(exc).__name__}: {exc}")
                rows_att.append(row)

        # ---- E3: alpha sweep (uint8 protocol), several attacks ------------
        sweep_suite = make_attack_suite(cfg["base_seed"] + 100 * idx + 50)
        for a in cfg["alphas"]:
            try:
                w, k = embed_watermark(host, logo, alpha=a)
```

```
                wq = _quantize(w)
                p_a, s_a = psnr(host, wq), ssim(host, wq)
            except Exception as exc:  # noqa: BLE001
                rows_sw.append({"image_id": image_id, "alpha": a, "attack": "(embed)", "status": "error",
                                "error": f"{type(exc).__name__}: {exc}"})
                continue
            for an in ["No Attack"] + list(cfg["sweep_attacks"]):
                row = {"image_id": image_id, "alpha": a, "attack": an, "psnr_db": p_a, "ssim": s_a,
                       "status": "ok", "error": ""}
                try:
                    ext = extract_watermark(sweep_suite[an](wq), k)
                    row["nc"] = normalized_correlation(logo, ext)
                    row["zncc"] = zero_normalized_correlation(logo, ext)
                    row["ber"] = bit_error_rate(logo, ext)
                except Exception as exc:  # noqa: BLE001
                    row.update(status="error", error=f"{type(exc).__name__}: {exc}")
                rows_sw.append(row)

        # ---- E4: controls ----------------------------------------------------
        def ctl(kind, other, ncv, berv, zz, extra=""):
            rows_ctl.append({"image_id": image_id, "control": kind, "other": other,
                             "nc": ncv, "ber": berv, "zncc": zz, "note": extra})

        # (genuine reference, no attack, uint8)
        ext = extract_watermark(wm_u8, key)
        ctl("genuine (own key, no attack)", image_id, normalized_correlation(logo, ext), bit_error_rate(logo,
ext), zero_normalized_correlation(logo, ext))
        # (d) degenerate: own unwatermarked host with own key
        ext = extract_watermark(host, key)
        ctl("d: own host unwatermarked, own key", image_id, normalized_correlation(logo, ext),
bit_error_rate(logo, ext), zero_normalized_correlation(logo, ext),
            extra="degenerate: S_att == S_original so Sw' = 0")
        for oid in others:
            oh = _load_host(data_dir, oid)
            ext = extract_watermark(oh, key)
            ctl("a: other host unwatermarked, key_i", oid, normalized_correlation(logo, ext),
bit_error_rate(logo, ext), zero_normalized_correlation(logo, ext))
            ow, _ = embed_watermark(oh, logo, alpha=alpha)
            ext = extract_watermark(_quantize(ow), key)
            ctl("b: other host watermarked, key_i", oid, normalized_correlation(logo, ext),
bit_error_rate(logo, ext), zero_normalized_correlation(logo, ext))
        for text in cfg["other_logos"]:
            logo_b = make_watermark_logo(cfg["logo_size"], text)
            wb, kb = embed_watermark(host, logo_b, alpha=alpha)
            wbq = _quantize(wb)
            ext_a = extract_watermark(wbq, key)       # decoded with logo-A key
            ext_b = extract_watermark(wbq, kb)        # decoded with its own key
            ctl("c: same host, other logo B, decoded with key A (NC vs A)", text,
                normalized_correlation(logo, ext_a), bit_error_rate(logo, ext_a),
zero_normalized_correlation(logo, ext_a))
            ctl("c-ref: same host, logo B, decoded with key B (NC vs B)", text,
                normalized_correlation(logo_b, ext_b), bit_error_rate(logo_b, ext_b),
zero_normalized_correlation(logo_b, ext_b))
    except Exception as exc:  # noqa: BLE001
        rec.update(status="error", error=f"{type(exc).__name__}: {exc}")
        LOG.exception("image %s failed", image_id)
    rec["peak_rss_mb"] = resource.getrusage(resource.RUSAGE_SELF).ru_maxrss / 1024.0
    return rec, rows_att, rows_sw, rows_ctl


# ---------------------------------------------------------------------------
# driver
# ---------------------------------------------------------------------------

def env_info():
    import PIL, pywt, scipy, skimage
    cpu = ""
    try:
        with open("/proc/cpuinfo") as f:
            cpu = next(l.split(":", 1)[1].strip() for l in f if l.startswith("model name"))
    except Exception:  # noqa: BLE001
        pass
    return {"python": sys.version.split()[0], "platform": platform.platform(), "cpu": cpu,
            "cpu_count": os.cpu_count(), "numpy": np.__version__, "scipy": scipy.__version__,
            "scikit-image": skimage.__version__, "PyWavelets": pywt.__version__, "Pillow": PIL.__version__,
            "pandas": pd.__version__}


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--config", default=None)
    ap.add_argument("--workers", type=int, default=os.cpu_count() or 1)
    ap.add_argument("--limit", type=int, default=None, help="process only the first N images (debug)")
```

```
    args = ap.parse_args()
    logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")

    cfg = dict(DEFAULT_CONFIG)
    if args.config:
        with open(args.config) as f:
            cfg.update(json.load(f))
    for k in ("data_dir", "results_dir", "outputs_dir"):
        cfg[k] = os.path.abspath(os.path.join(HERE, cfg[k]))
    os.makedirs(cfg["results_dir"], exist_ok=True)
    os.makedirs(os.path.join(cfg["outputs_dir"], "watermarked"), exist_ok=True)

    meta = pd.read_csv(os.path.join(cfg["data_dir"], "metadata.csv"))
    if args.limit:
        meta = meta.head(args.limit)
    ids = list(meta["image_id"])
    n = len(ids)
    rng = np.random.default_rng(cfg["base_seed"])
    tasks = []
    for i, iid in enumerate(ids):
        pool = [x for x in ids if x != iid]
        others = list(rng.choice(pool, size=min(cfg["n_controls_per_image"], len(pool)), replace=False))
        tasks.append((i, iid, cfg, others))

    LOG.info("processing %d images with %d worker(s)", n, args.workers)
    t_start = _t()
    results = []
    with cf.ProcessPoolExecutor(max_workers=args.workers) as ex:
        for res in ex.map(process_image, tasks):
            results.append(res)
            if len(results) % 10 == 0:
                LOG.info("  %d/%d images done", len(results), n)
    wall = _t() - t_start

    img_df = pd.DataFrame([r[0] for r in results]).sort_values("image_id")
    att_df = pd.DataFrame([row for r in results for row in r[1]])
    sw_df = pd.DataFrame([row for r in results for row in r[2]])
    ctl_df = pd.DataFrame([row for r in results for row in r[3]])
    rd = cfg["results_dir"]
    img_df.to_csv(os.path.join(rd, "results_images.csv"), index=False)
    att_df.to_csv(os.path.join(rd, "results_attacks.csv"), index=False)
    sw_df.to_csv(os.path.join(rd, "results_alpha_sweep.csv"), index=False)
    ctl_df.to_csv(os.path.join(rd, "results_controls.csv"), index=False)

    # processing status written back into the dataset metadata (merge; never drops rows)
    full = pd.read_csv(os.path.join(cfg["data_dir"], "metadata.csv"))
    st = img_df.set_index("image_id")["status"]
    att_ok = att_df.groupby("image_id")["status"].apply(lambda s: int((s == "ok").sum()))
    full["processing_status"] = full["image_id"].map(st).fillna(full.get("processing_status", "not processed"))
    full["attack_trials_ok"] = full["image_id"].map(att_ok).fillna(full.get("attack_trials_ok"))
    full["attack_trials_expected"] = len(ATTACK_NAMES) * len(cfg["protocols"])
    full.to_csv(os.path.join(cfg["data_dir"], "metadata.csv"), index=False)

    summary = {
        "config": {k: v for k, v in cfg.items() if not k.endswith("_dir")},
        "environment": env_info(),
        "images_requested": n,
        "images_ok": int((img_df["status"] == "ok").sum()),
        "watermarked_files_valid": int(img_df["output_valid"].fillna(False).sum()),
        "attack_trials_total": int(len(att_df)),
        "attack_trials_ok": int((att_df["status"] == "ok").sum()),
        "attack_trials_failed": int((att_df["status"] != "ok").sum()),
        "alpha_sweep_rows_ok": int((sw_df["status"] == "ok").sum()),
        "alpha_sweep_rows_total": int(len(sw_df)),
        "control_rows": int(len(ctl_df)),
        "wall_time_s": round(wall, 1),
        "workers": args.workers,
        "peak_rss_mb_max_over_workers": round(float(img_df["peak_rss_mb"].max()), 1),
    }
    with open(os.path.join(rd, "run_summary.json"), "w") as f:
        json.dump(summary, f, indent=2)
    LOG.info("summary: %s", json.dumps({k: v for k, v in summary.items() if k not in ("config",
"environment")}))
    return 0 if summary["attack_trials_failed"] == 0 and summary["images_ok"] == n else 1


if __name__ == "__main__":
    sys.exit(main())
```

A.9 analysis.py — Statistics, tables and figures from the recorded results (412 lines)

```
"""
analysis.py
```

```
===========
Statistics, tables and figures for the 100-image experiment.  Reads ONLY the
recorded CSV files in ``results/`` (written by run_batch_experiment.py) and the
dataset metadata - no number in the report is typed by hand.

    python analysis.py
"""

from __future__ import annotations

import os
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")

import json
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from PIL import Image
from scipy import stats

from attacks import ATTACK_FAMILY, ATTACK_NAMES, make_attack_suite
from logo import make_watermark_logo
from metrics import normalized_correlation, psnr, ssim, zero_normalized_correlation
from watermark_core import embed_watermark, extract_watermark

HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.abspath(os.path.join(HERE, "..", ".."))
RES = os.path.join(ROOT, "results")
TAB = os.path.join(RES, "tables")
VIS = os.path.join(ROOT, "visualizations")
DATA = os.path.join(ROOT, "data")
os.makedirs(TAB, exist_ok=True)
os.makedirs(VIS, exist_ok=True)

THR = 0.75
BLUE, ORANGE, AQUA, GREY, INK = "#2a78d6", "#eb6834", "#1baf7a", "#8a8a85", "#222222"
plt.rcParams.update({
    "font.size": 10, "axes.titlesize": 11, "axes.labelsize": 10, "axes.edgecolor": "#888888",
    "axes.spines.top": False, "axes.spines.right": False, "axes.grid": True, "grid.color": "#e4e4e0",
    "grid.linewidth": 0.7, "axes.axisbelow": True, "figure.facecolor": "white", "axes.facecolor": "white",
    "savefig.dpi": 200, "savefig.bbox": "tight", "legend.frameon": False,
})
rng = np.random.default_rng(20260713)


def wilson(k, n, z=1.96):
    if n == 0:
        return (np.nan, np.nan)
    p = k / n
    d = 1 + z * z / n
    c = (p + z * z / (2 * n)) / d
    h = z * np.sqrt(p * (1 - p) / n + z * z / (4 * n * n)) / d
    return c - h, c + h


def boot_ci(x, n=2000):
    x = np.asarray(x)
    means = rng.choice(x, size=(n, len(x)), replace=True).mean(axis=1)
    return np.percentile(means, [2.5, 97.5])


def save(fig, name):
    fig.savefig(os.path.join(VIS, name))
    plt.close(fig)


def main():
    meta = pd.read_csv(os.path.join(DATA, "metadata.csv"))
    img = pd.read_csv(os.path.join(RES, "results_images.csv"))
    att = pd.read_csv(os.path.join(RES, "results_attacks.csv"))
    sw = pd.read_csv(os.path.join(RES, "results_alpha_sweep.csv"))
    ctl = pd.read_csv(os.path.join(RES, "results_controls.csv"))
    summ = json.load(open(os.path.join(RES, "run_summary.json")))
    prim = summ["config"]["primary_protocol"]
    cat = meta.set_index("image_id")[["category", "is_synthetic", "mean_intensity", "contrast_std", "key"]]
    out = {}

    # ------------------------------------------------------------------ E1
    im = img.merge(meta[["image_id", "is_synthetic", "category"]], on="image_id")
    def desc(s):
        return dict(mean=s.mean(), std=s.std(), median=s.median(), min=s.min(), max=s.max())
```

```
    out["E1"] = {
        "n_images": int(len(im)),
        "psnr_all": desc(im.psnr_db), "ssim_all": desc(im.ssim),
        "psnr_real": desc(im[im.is_synthetic == 0].psnr_db), "ssim_real": desc(im[im.is_synthetic == 0].ssim),
        "psnr_synthetic": desc(im[im.is_synthetic == 1].psnr_db), "ssim_synthetic": desc(im[im.is_synthetic ==
1].ssim),
        "n_psnr_below_30": int((im.psnr_db < 30).sum()),
        "n_ssim_below_0.98": int((im.ssim < 0.98).sum()),
        "ssim_below_0.98_ids": list(im[im.ssim < 0.98].image_id),
        "psnr_float_mean": float(im.psnr_float_db.mean()),
        "max_abs_diff_max": float(im.max_abs_diff.max()), "mean_abs_diff_mean": float(im.mean_abs_diff.mean()),
        "t_embed_ms_median": float(im.t_embed_ms.median()), "key_mb": float(im.key_bytes.median() / 1e6),
    }
    t1 = pd.DataFrame({"PSNR (dB)": im.psnr_db.describe(), "SSIM": im.ssim.describe()}).round(4)
    t1.to_csv(os.path.join(TAB, "imperceptibility_summary.csv"))

    # ------------------------------------------------------------------ E2
    a = att[att.protocol == prim].merge(cat, left_on="image_id", right_index=True)
    rows = []
    for name in ATTACK_NAMES:
        x = a[a.attack == name]
        lo, hi = boot_ci(x.nc)
        k = int((x.nc >= THR).sum())
        wl, wh = wilson(k, len(x))
        real = x[x.is_synthetic == 0]
        rows.append(dict(attack=name, family=ATTACK_FAMILY[name], n=len(x), mean_nc=x.nc.mean(), ci_lo=lo,
ci_hi=hi,
                         median_nc=x.nc.median(), std_nc=x.nc.std(), min_nc=x.nc.min(), pass_rate=k / len(x),
                         mean_zncc=x.zncc.mean(), median_zncc=x.zncc.median(), zncc_pass_rate=(x.zncc >=
THR).mean(),
                         pass_lo=wl, pass_hi=wh, mean_ber=x.ber.mean(), mean_nc_real_only=real.nc.mean(),
                         pass_rate_real_only=(real.nc >= THR).mean(), t_attack_ms=x.t_attack_ms.median(),
                         t_extract_ms=x.t_extract_ms.median()))
    rob = pd.DataFrame(rows)
    rob.to_csv(os.path.join(TAB, "robustness_by_attack.csv"), index=False)
    fam = a.groupby("family").agg(mean_nc=("nc", "mean"), pass_rate=("nc", lambda s: (s >= THR).mean()),
n=("nc", "size")).reset_index()
    fam.to_csv(os.path.join(TAB, "robustness_by_family.csv"), index=False)

    other = att[att.protocol != prim].merge(cat, left_on="image_id", right_index=True)
    cmp_rows = []
    for name in ATTACK_NAMES:
        u = a[a.attack == name].set_index("image_id").nc
        f = other[other.attack == name].set_index("image_id").nc
        cmp_rows.append(dict(attack=name, mean_nc_uint8=u.mean(), mean_nc_float=f.mean(),
                             max_abs_diff_per_image=(u - f.reindex(u.index)).abs().max()))
    pd.DataFrame(cmp_rows).to_csv(os.path.join(TAB, "protocol_comparison.csv"), index=False)

    out["E2"] = {
        "protocol": prim, "n_trials": int(len(a)), "mean_nc": float(a.nc.mean()),
        "n_trials_attacks_only": int((a.attack != "No Attack").sum()),
        "n_pass_attacks_only": int(((a.attack != "No Attack") & (a.nc >= THR)).sum()),
        "pass_rate_attacks_only": float((a[a.attack != "No Attack"].nc >= THR).mean()),
        "mean_nc_attacks_only": float(a[a.attack != "No Attack"].nc.mean()),
        "pass_rate": float((a.nc >= THR).mean()), "n_pass": int((a.nc >= THR).sum()),
        "pass_rate_real_only": float((a[a.is_synthetic == 0].nc >= THR).mean()),
        "mean_nc_real_only": float(a[a.is_synthetic == 0].nc.mean()),
        "mean_nc_float_protocol": float(other.nc.mean()), "pass_rate_float_protocol": float((other.nc >=
THR).mean()),
        "mean_ber": float(a.ber.mean()), "mean_zncc": float(a.zncc.mean()), "zncc_pass_rate": float((a.zncc >=
THR).mean()),
        "zncc_clean_mean": float(a[a.attack == "No Attack"].zncc.mean()),
        "clean_nc_mean": float(a[a.attack == "No Attack"].nc.mean()), "clean_nc_min": float(a[a.attack == "No
Attack"].nc.min()),
        "attacks_with_pass_rate_ge_0.95": list(rob[rob.pass_rate >= 0.95].attack),
        "weakest": rob.sort_values("pass_rate").head(3)[["attack", "pass_rate", "mean_nc"]].to_dict("records"),
    }

    # correlations with image statistics (Spearman) -------------------------
    piv = a.pivot(index="image_id", columns="attack", values="nc").join(cat)
    cor = []
    for name in ATTACK_NAMES:
        if name == "No Attack":
            continue
        r1 = stats.spearmanr(piv["contrast_std"], piv[name])
        r2 = stats.spearmanr(piv["mean_intensity"], piv[name])
        cor.append(dict(attack=name, rho_contrast=r1[0], p_contrast=r1[1], rho_mean_intensity=r2[0],
p_mean_intensity=r2[1]))
    pd.DataFrame(cor).round(4).to_csv(os.path.join(TAB, "nc_vs_image_statistics.csv"), index=False)

    # category x attack-family heat table -----------------------------------
    ct = a.groupby(["category", "family"]).nc.mean().unstack().round(3)
    ct["n_images"] = meta.groupby("category").size()
```

```
    ct.to_csv(os.path.join(TAB, "nc_by_category_family.csv"))
    ca = a.groupby(["category", "attack"]).nc.mean().unstack().round(3)
    ca.to_csv(os.path.join(TAB, "nc_by_category_attack.csv"))

    # ------------------------------------------------------------------ E3
    swo = sw[sw.status == "ok"]
    s = swo.groupby(["attack", "alpha"]).agg(psnr=("psnr_db", "mean"), psnr_sd=("psnr_db", "std"),
ssim=("ssim", "mean"),
                                             nc_mean=("nc", "mean"), nc_median=("nc", "median"),
                                             nc_q25=("nc", lambda x: x.quantile(.25)), nc_q75=("nc", lambda x:
x.quantile(.75)),
                                             zncc_mean=("zncc", "mean"), ber_mean=("ber", "mean"),
                                             pass_rate=("nc", lambda x: (x >= THR).mean())).reset_index()
    s.round(4).to_csv(os.path.join(TAB, "alpha_sweep_summary.csv"), index=False)
    out["E3"] = {"table": s.round(4).to_dict("records")}
    # metric reference points for this logo
    logo0 = make_watermark_logo(summ["config"]["logo_size"], summ["config"]["logo_text"])
    U_, S_, Vt_ = np.linalg.svd(logo0)
    rank1 = np.clip((U_[:, :1] * S_[:1]) @ Vt_[:1], 0, 255)
    out["metric_baselines"] = {
        "logo_fill_ratio": float((logo0 > 0).mean()),
        "nc_constant_white_image": normalized_correlation(logo0, np.full_like(logo0, 255.0)),
        "nc_rank1_approximation": normalized_correlation(logo0, rank1),
        "zncc_rank1_approximation": zero_normalized_correlation(logo0, rank1),
        "sigma1_energy_fraction": float(S_[0] ** 2 / (S_ ** 2).sum()),
    }

    # ------------------------------------------------------------------ E4
    def sel(prefix):
        return ctl[ctl.control.str.startswith(prefix)].nc
    g = sel("genuine")
    crow = []
    for lbl, pre in [("genuine: own key, clean watermarked image", "genuine"),
                     ("a: other host, UNwatermarked, key of image i", "a:"),
                     ("b: other host, watermarked, key of image i", "b:"),
                     ("c: same host, different logo embedded, decoded with key A (NC vs logo A)", "c: "),
                     ("c-ref: same host, logo B decoded with its own key (NC vs logo B)", "c-ref"),
                     ("d: own host unwatermarked, own key (degenerate)", "d:")]:
        x = sel(pre)
        z = ctl[ctl.control.str.startswith(pre)].zncc
        crow.append(dict(control=lbl, n=len(x), mean_nc=x.mean(), median=x.median(), p95=x.quantile(.95),
max=x.max(),
                         frac_ge_0_75=(x >= THR).mean(), frac_ge_0_90=(x >= .9).mean(),
                         mean_zncc=z.mean(), p95_zncc=z.quantile(.95), max_zncc=z.max(), frac_zncc_ge_0_75=(z
>= THR).mean()))
    pd.DataFrame(crow).round(4).to_csv(os.path.join(TAB, "controls_summary.csv"), index=False)
    auc = {}
    for k, pre in [("a", "a:"), ("b", "b:"), ("c", "c: ")]:
        x = sel(pre)
        u = stats.mannwhitneyu(g, x, alternative="greater")
        auc[k] = float(u.statistic / (len(g) * len(x)))
    out["E4"] = {"summary": pd.DataFrame(crow).round(4).to_dict("records"), "auc_genuine_vs_control": auc}

    # attacked genuine vs wrong-image controls: how much of the attack range is separable by 0.75?
    wrong_p95 = float(pd.concat([sel("a:"), sel("b:")]).quantile(.95))
    out["E4"]["wrong_image_p95"] = wrong_p95
    out["E4"]["attacked_genuine_below_wrong_p95"] = float((a.nc < wrong_p95).mean())

    # timing ------------------------------------------------------------------
    tim = a.groupby("attack").agg(t_attack_ms=("t_attack_ms", "median"), t_extract_ms=("t_extract_ms",
"median")).reset_index()
    tim.round(3).to_csv(os.path.join(TAB, "timing_by_attack.csv"), index=False)
    out["timing"] = {"embed_ms_median": float(img.t_embed_ms.median()),
                     "extract_ms_median": float(a.t_extract_ms.median()),
                     "throughput_embed_per_s_single_core": float(1000 / img.t_embed_ms.median()),
                     "peak_rss_mb": float(img.peak_rss_mb.max())}

    json.dump(out, open(os.path.join(RES, "analysis_summary.json"), "w"), indent=2, default=float)

    # ================================================================ FIGURES
    # Fig: imperceptibility distributions
    fig, ax = plt.subplots(1, 2, figsize=(9, 3.3))
    real = im[im.is_synthetic == 0]; syn = im[im.is_synthetic == 1]
    ax[0].hist(real.psnr_db, bins=np.arange(35.4, 44.2, 0.2), color=BLUE, label=f"real (n={len(real)})")
    ax[0].hist(syn.psnr_db, bins=np.arange(35.4, 44.2, 0.2), color=ORANGE, label=f"synthetic (n={len(syn)})")
    ax[0].axvline(30, color=INK, lw=1, ls="--"); ax[0].text(30.1, ax[0].get_ylim()[1] * .9, "30 dB",
fontsize=8)
    ax[0].set_xlabel("PSNR of watermarked image (dB)"); ax[0].set_ylabel("images"); ax[0].legend()
    ax[1].hist(real.ssim, bins=np.linspace(0.55, 1.0, 46), color=BLUE)
    ax[1].hist(syn.ssim, bins=np.linspace(0.55, 1.0, 46), color=ORANGE)
    ax[1].axvline(0.98, color=INK, lw=1, ls="--"); ax[1].text(0.9805, ax[1].get_ylim()[1] * .9, "0.98",
fontsize=8)
    ax[1].set_xlabel("SSIM of watermarked image"); ax[1].set_ylabel("images")
```

```
    fig.suptitle(f"Imperceptibility at alpha = {summ['config']['alpha']} across {len(im)} images", y=1.02,
fontsize=11)
    fig.tight_layout(); save(fig, "fig_imperceptibility_hist.png")

    # Fig: NC distribution per attack (box + pass-rate)
    order = rob.sort_values("median_nc")["attack"].tolist()
    fig, ax = plt.subplots(figsize=(8.6, 6.2))
    data = [a[a.attack == n].nc.values for n in order]
    bp = ax.boxplot(data, orientation="horizontal", patch_artist=True, widths=0.6, showfliers=True,
                    flierprops=dict(marker="o", markersize=2.5, markerfacecolor=GREY, markeredgecolor="none",
alpha=.6),
                    medianprops=dict(color=INK, lw=1.4), whiskerprops=dict(color=GREY),
capprops=dict(color=GREY),
                    boxprops=dict(edgecolor=GREY))
    for patch, n in zip(bp["boxes"], order):
        patch.set_facecolor(BLUE if rob.set_index("attack").loc[n, "median_nc"] >= THR else ORANGE)
        patch.set_alpha(.85)
    ax.set_yticklabels(order)
    ax.axvline(THR, color=INK, ls="--", lw=1)
    ax.text(THR - .005, len(order) + 0.55, "threshold 0.75", fontsize=8, ha="right", va="center")
    for i, n in enumerate(order, 1):
        pr = rob.set_index("attack").loc[n, "pass_rate"]
        ax.text(1.03, i, f"{pr*100:.0f}%", va="center", fontsize=8.5, color=INK)
    ax.text(1.03, len(order) + 0.55, "NC >= 0.75", fontsize=8, color=INK, va="center")
    ax.set_xlim(-0.02, 1.12); ax.set_xlabel("Normalized correlation (NC) of extracted watermark, 100 images")
    ax.set_title("Robustness per attack: NC over 100 images (box = IQR, line = median)", loc="left")
    ax.grid(axis="y", visible=False)
    fig.tight_layout(); save(fig, "fig_nc_boxplot.png")

    # Fig: pass-rate bars with Wilson CI
    r = rob.sort_values("pass_rate")
    fig, ax = plt.subplots(figsize=(8, 5.4))
    ax.barh(r.attack, r.pass_rate * 100, color=[BLUE if v >= .9 else ORANGE for v in r.pass_rate], height=.62)
    ax.errorbar(r.pass_rate * 100, np.arange(len(r)), xerr=[np.clip((r.pass_rate - r.pass_lo) * 100, 0, None),
np.clip((r.pass_hi - r.pass_rate) * 100, 0, None)],
                fmt="none", ecolor=INK, elinewidth=1, capsize=2)
    for i, v in enumerate(r.pass_rate):
        ax.text(min(v * 100 + 3.5, 104), i, f"{v*100:.0f}%", va="center", fontsize=8.5)
    ax.set_xlim(0, 112); ax.set_xlabel("Images with NC >= 0.75 (%)  - error bars: 95% Wilson interval")
    ax.set_title("Detection pass rate per attack (n = 100 images each)")
    ax.grid(axis="y", visible=False)
    fig.tight_layout(); save(fig, "fig_pass_rate.png")

    # Fig: alpha trade-off - small multiples, one axis per panel (no dual axis)
    fig, ax = plt.subplots(1, 3, figsize=(11.5, 3.5))
    p0 = s[s.attack == "No Attack"]
    ax[0].plot(p0.alpha, p0.psnr, color=BLUE, marker="o", ms=4, lw=1.6)
    ax[0].fill_between(p0.alpha, p0.psnr - p0.psnr_sd, p0.psnr + p0.psnr_sd, color=BLUE, alpha=.15, lw=0)
    ax[0].axhline(30, color=INK, lw=1, ls="--"); ax[0].set_ylabel("mean PSNR (dB)")
    ax[0].set_title("Imperceptibility (band = +-1 sd)")
    cols = {"JPEG Q=50": BLUE, "Gaussian Noise (var=0.02)": ORANGE, "Gaussian Blur (sigma=1.5)": AQUA,
"Rotation 5 deg": GREY}
    for an, c in cols.items():
        q = s[s.attack == an]
        ax[1].plot(q.alpha, q.nc_mean, color=c, marker="o", ms=3.5, lw=1.5, label=an)
        ax[2].plot(q.alpha, q.pass_rate * 100, color=c, marker="o", ms=3.5, lw=1.5, label=an)
    ax[1].axhline(THR, color=INK, lw=1, ls="--"); ax[1].set_ylim(0.4, 1.02); ax[1].set_ylabel("mean NC (100
images)")
    ax[1].set_title("Robustness: mean NC after attack"); ax[1].legend(fontsize=7.5, loc="lower right")
    ax[2].set_ylim(0, 102); ax[2].set_ylabel("images with NC >= 0.75 (%)"); ax[2].set_title("Robustness:
detection pass rate")
    for x_ in ax:
        x_.set_xlabel("embedding strength alpha")
    fig.tight_layout(); save(fig, "fig_alpha_tradeoff.png")

    # Fig: NC vs ZNCC per attack
    fig, ax = plt.subplots(figsize=(8, 5.2))
    rr = rob.sort_values("mean_nc")
    yy = np.arange(len(rr))
    ax.barh(yy + .18, rr.mean_nc, height=.34, color=BLUE, label="NC (original metric)")
    ax.barh(yy - .18, rr.mean_zncc, height=.34, color=ORANGE, label="ZNCC (mean-centred)")
    ax.set_yticks(yy); ax.set_yticklabels(rr.attack); ax.axvline(THR, color=INK, ls="--", lw=1)
    ax.set_xlabel("mean over 100 images"); ax.legend(loc="lower right"); ax.grid(axis="y", visible=False)
    ax.set_title("Plain NC is optimistic: ZNCC removes the brightness offset")
    fig.tight_layout(); save(fig, "fig_nc_vs_zncc.png")

    # Fig: controls
    fig, ax = plt.subplots(figsize=(8, 4))
    bins = np.linspace(0, 1, 41)
    ax.hist(sel("a:"), bins=bins, alpha=.85, color=ORANGE, label="a: other host, unwatermarked", density=True)
    ax.hist(sel("b:"), bins=bins, alpha=.55, color=AQUA, label="b: other host, watermarked (wrong key)",
density=True)
    ax.hist(g, bins=bins, alpha=.75, color=BLUE, label="genuine: own key", density=True)
```

```
    ax.axvline(THR, color=INK, ls="--", lw=1)
    ax.set_xlabel("NC against the owner's logo"); ax.set_ylabel("density")
    ax.set_title("Control experiments: wrong-image/wrong-key NC overlaps the 0.75 threshold")
    ax.legend(loc="upper center", fontsize=8)
    fig.tight_layout(); save(fig, "fig_controls_hist.png")

    # Fig: FPP bar (wrong logo decoded with the key-A)
    cc = sel("c: "); cr = sel("c-ref")
    fig, ax = plt.subplots(figsize=(6.4, 3.4))
    vals = [g.mean(), cc.mean(), cr.mean(), sel("a:").mean(), sel("b:").mean()]
    labs = ["genuine\n(logo A, key A)", "logo B embedded,\ndecoded with key A", "logo B embedded,\ndecoded with
key B", "other host,\nunwatermarked", "other host,\nwrong key"]
    ax.bar(labs, vals, color=[BLUE, ORANGE, BLUE, GREY, GREY], width=.6)
    for i, v in enumerate(vals):
        ax.text(i, v + .015, f"{v:.3f}", ha="center", fontsize=9)
    ax.set_ylim(0, 1.12); ax.set_ylabel("mean NC vs the logo tested")
    ax.set_title("False-positive problem: the wrong logo is reported as the owner's logo")
    ax.tick_params(axis="x", labelsize=8); ax.grid(axis="x", visible=False)
    fig.tight_layout(); save(fig, "fig_false_positive.png")

    # Fig: category x family heatmap
    hm = ct.drop(columns="n_images")
    hm = hm.loc[hm.index != "synthetic_control"]
    fig, ax = plt.subplots(figsize=(8.4, 6.2))
    im_ = ax.imshow(hm.values, cmap="Blues", vmin=0, vmax=1, aspect="auto")
    ax.set_xticks(range(hm.shape[1])); ax.set_xticklabels(hm.columns, rotation=30, ha="right")
    ax.set_yticks(range(hm.shape[0])); ax.set_yticklabels([f"{c} ({int(ct.loc[c,'n_images'])})" for c in
hm.index])
    for i in range(hm.shape[0]):
        for j in range(hm.shape[1]):
            v = hm.values[i, j]
            ax.text(j, i, f"{v:.2f}", ha="center", va="center", fontsize=8, color="white" if v > .62 else INK)
    ax.grid(False); fig.colorbar(im_, ax=ax, fraction=.04, label="mean NC")
    ax.set_title("Mean NC by image category and attack family (real images)")
    fig.tight_layout(); save(fig, "fig_category_heatmap.png")

    # Fig: NC vs contrast scatter for 3 attacks
    fig, ax = plt.subplots(1, 3, figsize=(10.5, 3.3), sharey=True)
    for k, name in enumerate(["Gaussian Noise (var=0.02)", "Rotation 15 deg", "Histogram Equalization"]):
        d = piv
        ax[k].scatter(d["contrast_std"], d[name], s=14, c=np.where(d["is_synthetic"] == 1, ORANGE, BLUE),
alpha=.8, lw=0)
        rho = stats.spearmanr(d["contrast_std"], d[name])[0]
        ax[k].set_title(f"{name}\nSpearman rho = {rho:.2f}", fontsize=9.5)
        ax[k].set_xlabel("image contrast (std of grey levels)"); ax[k].axhline(THR, color=INK, ls="--", lw=1)
    ax[0].set_ylabel("NC after attack")
    fig.tight_layout(); save(fig, "fig_nc_vs_contrast.png")

    # Fig: timing
    fig, ax = plt.subplots(figsize=(6.5, 3.2))
    ax.hist(img.t_embed_ms, bins=20, color=BLUE, label=f"embed (median {img.t_embed_ms.median():.0f} ms)")
    ax.hist(a.t_extract_ms, bins=40, color=ORANGE, alpha=.8, label=f"extract (median
{a.t_extract_ms.median():.0f} ms)")
    ax.set_xlabel("time per 512x512 image (ms, single thread)"); ax.set_ylabel("operations"); ax.legend()
    fig.tight_layout(); save(fig, "fig_timing.png")

    # ------------------------------------------------- qualitative figures
    cfg = summ["config"]; alpha = cfg["alpha"]
    logo = make_watermark_logo(cfg["logo_size"], cfg["logo_text"])
    Image.fromarray(logo.astype(np.uint8)).save(os.path.join(VIS, "watermark_logo.png"))
    reps = ["image_066", "image_064", "image_001", "image_051", "image_055", "image_100"]
    fig, axes = plt.subplots(len(reps), 3, figsize=(8.4, 2.8 * len(reps)))
    for r_i, iid in enumerate(reps):
        host = np.asarray(Image.open(os.path.join(DATA, "original", iid + ".png")), dtype=np.float64)
        w = np.asarray(Image.open(os.path.join(ROOT, "outputs", "watermarked", iid + "_wm.png")),
dtype=np.float64)
        d = np.abs(host - w)
        row = img.set_index("image_id").loc[iid]
        key = meta.set_index("image_id").loc[iid, "key"]
        axes[r_i, 0].imshow(host, cmap="gray", vmin=0, vmax=255); axes[r_i, 0].set_title(f"{iid} ({key})
original", fontsize=8.5)
        axes[r_i, 1].imshow(w, cmap="gray", vmin=0, vmax=255); axes[r_i, 1].set_title(f"watermarked  PSNR
{row.psnr_db:.1f} dB, SSIM {row.ssim:.3f}", fontsize=8.5)
        im2 = axes[r_i, 2].imshow(np.clip(d * 10, 0, 255), cmap="magma", vmin=0, vmax=255); axes[r_i,
2].set_title(f"|difference| x10 (max {d.max():.0f} grey levels)", fontsize=8.5)
        for x in axes[r_i]:
            x.axis("off")
    fig.tight_layout(); save(fig, "fig_representative_comparison.png")

    # extraction gallery for the cameraman image under the 14 attacks and the unattacked reference
    iid = "image_066"
    host = np.asarray(Image.open(os.path.join(DATA, "original", iid + ".png")), dtype=np.float64)
    w, key = embed_watermark(host, logo, alpha=alpha)
```

```
    wq = np.clip(np.rint(w), 0, 255)
    suite = make_attack_suite(cfg["base_seed"] + 100 * (int(iid[-3:]) - 1))
    fig, axes = plt.subplots(3, 6, figsize=(12, 6.6))
    axes[0, 0].imshow(logo, cmap="gray"); axes[0, 0].set_title("original logo", fontsize=8.5)
    for ax_, name in zip(axes.flat[1:], ATTACK_NAMES):
        ext = extract_watermark(suite[name](wq), key)
        ax_.imshow(ext, cmap="gray", vmin=0, vmax=255)
        ax_.set_title(f"{name}\nNC={normalized_correlation(logo, ext):.3f}", fontsize=7.8)
    for ax_ in axes.flat[16:]:
        ax_.axis("off")
    for ax_ in axes.flat:
        ax_.set_xticks([]); ax_.set_yticks([]); ax_.grid(False)
    fig.suptitle("Extracted watermark under each attack (image_066, cameraman)", y=1.0, fontsize=11)
    fig.tight_layout(); save(fig, "fig_extraction_gallery.png")

    # contact sheet of watermarked outputs
    T = 128
    sheet = Image.new("L", (10 * T, 10 * T), 255)
    for i, iid in enumerate(meta.image_id):
        sheet.paste(Image.open(os.path.join(ROOT, "outputs", "watermarked", iid + "_wm.png")).resize((T, T)),
((i % 10) * T, (i // 10) * T))
    sheet.save(os.path.join(VIS, "contact_sheet_watermarked.png"))
    print(json.dumps({k: out[k] for k in ("E1", "E2")}, indent=1, default=float)[:3000])


if __name__ == "__main__":
    main()
```

A.10 benchmark_tech.py — Library and scheme benchmarks (S1, S2) (467 lines)

```
"""
benchmark_tech.py
=================
Controlled comparison of the project against established technologies.

Part A  (component level)  - the *same* operation executed with different
        libraries on identical input: DCT, SVD, Haar DWT, JPEG round-trip,
        median / Gaussian filtering, rotation, rescaling, histogram
        equalisation, PSNR.  Reports median time (warm-up, repeats, single
        thread, in-memory, I/O excluded) and numerical agreement with the
        project's own implementation.
Part B  (scheme level)     - the project's global DWT-DCT-SVD scheme against
        (i) the open-source `blind-watermark` package (block DWT-DCT-SVD, blind),
        (ii) a spatial LSB baseline and (iii) an 8x8 block-DCT baseline, all
        carrying the same 4095-bit payload (the 64x64 logo, last bit dropped
        because blind-watermark needs strictly fewer bits than its 4096 blocks).

    python benchmark_tech.py            # writes results/tech_*.csv, tech_summary.json, figures
"""

from __future__ import annotations

import os
for _v in ("OPENBLAS_NUM_THREADS", "OMP_NUM_THREADS", "MKL_NUM_THREADS"):
    os.environ.setdefault(_v, "1")

import io
import json
import shutil
import subprocess
import sys
import tempfile
import time
import warnings

import cv2
import numpy as np
import pandas as pd
import pywt
import scipy
import scipy.fft
import scipy.linalg
import skimage
from PIL import Image, ImageFilter
from scipy import ndimage
from scipy.fftpack import dct as _dct
from skimage import exposure, filters as skfilters, transform as sktransform
from skimage.metrics import peak_signal_noise_ratio as sk_psnr

from attacks import make_attack_suite
from logo import make_watermark_logo
from metrics import psnr
from watermark_core import dct2, embed_watermark, extract_watermark
```

```
warnings.filterwarnings("ignore")
cv2.setNumThreads(1)
HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.abspath(os.path.join(HERE, "..", ".."))
RES = os.path.join(ROOT, "results")
VIS = os.path.join(ROOT, "visualizations")
DATA = os.path.join(ROOT, "data")


def load(iid):
    return np.asarray(Image.open(os.path.join(DATA, "original", iid + ".png")), dtype=np.float64)


def tmed(fn, repeats=9, warm=2):
    for _ in range(warm):
        fn()
    ts = []
    for _ in range(repeats):
        t = time.perf_counter()
        fn()
        ts.append((time.perf_counter() - t) * 1e3)
    return float(np.median(ts)), float(np.percentile(ts, 25)), float(np.percentile(ts, 75))


def have(cmd):
    return shutil.which(cmd) is not None


# ---------------------------------------------------------------------------
# Part A: component-level
# ---------------------------------------------------------------------------

def dct_matrix(n):
    k = np.arange(n)[:, None]; i = np.arange(n)[None, :]
    c = np.sqrt(2.0 / n) * np.cos(np.pi * (2 * i + 1) * k / (2 * n))
    c[0] /= np.sqrt(2)
    return c


def haar_numpy(a):
    a00, a01, a10, a11 = a[0::2, 0::2], a[0::2, 1::2], a[1::2, 0::2], a[1::2, 1::2]
    return (a00 + a01 + a10 + a11) / 2.0


def jpeg_pillow(a, q=50):
    b = io.BytesIO(); Image.fromarray(a.astype(np.uint8)).save(b, "JPEG", quality=q); n = b.tell(); b.seek(0)
    return np.array(Image.open(b)).astype(np.float64), n


def jpeg_cv2(a, q=50):
    ok, buf = cv2.imencode(".jpg", a.astype(np.uint8), [cv2.IMWRITE_JPEG_QUALITY, q])
    return cv2.imdecode(buf, cv2.IMREAD_GRAYSCALE).astype(np.float64), len(buf)


def jpeg_magick(a, q=50, tmp=None):
    src, dst = os.path.join(tmp, "in.png"), os.path.join(tmp, "out.jpg")
    Image.fromarray(a.astype(np.uint8)).save(src)
    subprocess.run(["convert", src, "-quality", str(q), dst], check=True)
    return np.array(Image.open(dst).convert("L")).astype(np.float64), os.path.getsize(dst)


def rot_skimage(a):
    return sktransform.rotate(a / 255.0, 15, resize=False, mode="edge") * 255.0


def rot_cv2(a):
    h, w = a.shape
    M = cv2.getRotationMatrix2D(((w - 1) / 2, (h - 1) / 2), 15, 1.0)
    return cv2.warpAffine(a, M, (w, h), flags=cv2.INTER_LINEAR, borderMode=cv2.BORDER_REPLICATE)


def rot_pillow(a):
    return np.array(Image.fromarray(a.astype(np.uint8)).rotate(15, resample=Image.BILINEAR)).astype(np.float64)


def scale_skimage(a):
    s = sktransform.resize(a, (a.shape[0] // 2, a.shape[1] // 2), anti_aliasing=True, preserve_range=True)
    return sktransform.resize(s, a.shape, anti_aliasing=True, preserve_range=True)


def scale_cv2(a):
    s = cv2.resize(a, (a.shape[1] // 2, a.shape[0] // 2), interpolation=cv2.INTER_AREA)
```

```
    return cv2.resize(s, (a.shape[1], a.shape[0]), interpolation=cv2.INTER_LINEAR)


def scale_pillow(a):
    im = Image.fromarray(a.astype(np.uint8))
    s = im.resize((a.shape[1] // 2, a.shape[0] // 2), Image.BILINEAR, reducing_gap=None)
    return np.array(s.resize(im.size, Image.BILINEAR)).astype(np.float64)


def part_a(images):
    tmp = tempfile.mkdtemp()
    rows = []
    h512 = images[0]
    ll = h512[:256, :256].copy()
    u8 = h512.astype(np.uint8)
    C = dct_matrix(256)

    ops = {
        "2-D DCT-II 256x256": [
            ("scipy.fftpack.dct (project)", lambda: dct2(ll), "ref"),
            ("scipy.fft.dctn", lambda: scipy.fft.dctn(ll, norm="ortho"), ""),
            ("cv2.dct", lambda: cv2.dct(ll), ""),
            ("NumPy matrix C.A.C^T", lambda: C @ ll @ C.T, ""),
        ],
        "SVD 256x256": [
            ("numpy.linalg.svd (project)", lambda: np.linalg.svd(ll, full_matrices=False)[1], "ref"),
            ("scipy.linalg.svd (gesdd)", lambda: scipy.linalg.svd(ll, full_matrices=False, check_finite=False)
[1], ""),
            ("scipy.linalg.svd (gesvd)", lambda: scipy.linalg.svd(ll, full_matrices=False,
lapack_driver="gesvd", check_finite=False)[1], ""),
            ("cv2.SVDecomp", lambda: cv2.SVDecomp(ll)[0].ravel(), ""),
        ],
        "Haar DWT level-1 (LL) 512x512": [
            ("PyWavelets dwt2 (project)", lambda: pywt.dwt2(h512, "haar")[0], "ref"),
            ("NumPy slicing", lambda: haar_numpy(h512), ""),
        ],
        "JPEG round-trip Q=50, 512x512": [
            ("Pillow (project)", lambda: jpeg_pillow(h512)[0], "ref"),
            ("OpenCV imencode/imdecode", lambda: jpeg_cv2(h512)[0], ""),
        ] + ([("ImageMagick convert (process + file I/O)", lambda: jpeg_magick(h512, tmp=tmp)[0], "io")] if
have("convert") else []),
        "Median filter 3x3": [
            ("scipy.ndimage (project)", lambda: ndimage.median_filter(h512, size=3), "ref"),
            ("cv2.medianBlur (uint8)", lambda: cv2.medianBlur(u8, 3), ""),
            ("skimage.filters.median (uint8)", lambda: skfilters.median(u8, np.ones((3, 3), bool)), ""),
        ],
        "Gaussian blur sigma=1.5": [
            ("scipy.ndimage (project)", lambda: ndimage.gaussian_filter(h512, 1.5), "ref"),
            ("cv2.GaussianBlur", lambda: cv2.GaussianBlur(h512, (0, 0), 1.5, borderType=cv2.BORDER_REFLECT),
""),
            ("skimage.filters.gaussian", lambda: skfilters.gaussian(h512, sigma=1.5, preserve_range=True), ""),
        ],
        "Rotation 15 deg (bilinear)": [
            ("skimage.transform.rotate (project)", lambda: rot_skimage(h512), "ref"),
            ("cv2.warpAffine", lambda: rot_cv2(h512), ""),
            ("Pillow Image.rotate", lambda: rot_pillow(h512), ""),
        ],
        "Down/up-scale 0.5x": [
            ("skimage.transform.resize (project)", lambda: scale_skimage(h512), "ref"),
            ("cv2.resize (AREA down, LINEAR up)", lambda: scale_cv2(h512), ""),
            ("Pillow resize (BILINEAR)", lambda: scale_pillow(h512), ""),
        ],
        "Histogram equalisation": [
            ("skimage.exposure.equalize_hist (project)", lambda: exposure.equalize_hist(h512 / 255.0) * 255.0,
"ref"),
            ("cv2.equalizeHist (uint8)", lambda: cv2.equalizeHist(u8).astype(np.float64), ""),
        ],
        "PSNR 512x512": [
            ("skimage.metrics (project)", lambda: sk_psnr(h512, h512 + 1.0, data_range=255), "ref"),
            ("cv2.PSNR", lambda: cv2.PSNR(h512, h512 + 1.0), ""),
            ("NumPy formula", lambda: 10 * np.log10(255 ** 2 / np.mean((h512 - (h512 + 1.0)) ** 2)), ""),
        ],
    }
    for op, variants in ops.items():
        ref = np.asarray(variants[0][1]())
        for label, fn, tag in variants:
            out = np.asarray(fn())
            med, q1, q3 = tmed(fn)
            row = dict(operation=op, implementation=label, median_ms=med, q25_ms=q1, q75_ms=q3,
                       includes_process_and_io=(tag == "io"))
            if out.shape == ref.shape and out.size > 1:
                if op.startswith("Rotation"):
```

```
                    sl = (slice(128, 384), slice(128, 384))   # compare the interior only (border handling
differs)
                    d = out[sl] - ref[sl]
                    row["agreement"] = "PSNR %.1f dB (interior)" % (10 * np.log10(255 ** 2 / max(np.mean(d **
2), 1e-12)))
                elif op.startswith("SVD") or op.startswith("2-D DCT"):
                    scale = max(np.abs(ref).max(), 1e-12)
                    row["agreement"] = "max rel. diff %.1e" % (np.abs(out - ref).max() / scale)
                elif op.startswith("Haar"):
                    row["agreement"] = "max abs diff %.1e" % np.abs(out - ref).max()
                else:
                    d = out.astype(np.float64) - ref.astype(np.float64)
                    mse = np.mean(d ** 2)
                    row["agreement"] = "identical" if mse == 0 else "PSNR %.1f dB" % (10 * np.log10(255 ** 2 /
mse))
            else:
                row["agreement"] = "%.4f vs %.4f" % (float(np.ravel(out)[0]), float(np.ravel(ref)[0]))
            rows.append(row)
    # extra facts for JPEG
    j = {"Pillow": jpeg_pillow(h512)[1], "OpenCV": jpeg_cv2(h512)[1]}
    if have("convert"):
        j["ImageMagick"] = jpeg_magick(h512, tmp=tmp)[1]
    shutil.rmtree(tmp, ignore_errors=True)
    return pd.DataFrame(rows), j


# ---------------------------------------------------------------------------
# Part B: scheme-level
# ---------------------------------------------------------------------------

NB = 4095  # payload bits


def bits_to_logo(bits):
    b = np.zeros(4096); b[:NB] = np.asarray(bits[:NB], dtype=float)
    return b.reshape(64, 64) * 255.0


class Scheme:
    name = ""
    blind = None
    needs = ""


class OursScheme(Scheme):
    name = "Project: global DWT-DCT-SVD"
    blind = False
    needs = "owner's key (S_orig, Uw, Vw), ~2.1 MB per image"

    def __init__(self, alpha=0.10):
        self.alpha = alpha
        self.logo = make_watermark_logo()

    def embed(self, host):
        w, self.key = embed_watermark(host, self.logo, alpha=self.alpha)
        return w

    def extract(self, img):
        return extract_watermark(img, self.key) > 127.5


class LSBScheme(Scheme):
    name = "Baseline: spatial LSB"
    blind = True
    needs = "payload length only"

    def __init__(self, bits):
        self.bits = np.asarray(bits[:NB], dtype=np.uint8)

    def embed(self, host):
        flat = np.rint(host).astype(np.uint8).flatten()
        flat[:NB] = (flat[:NB] & 0xFE) | self.bits
        return flat.reshape(host.shape).astype(np.float64)

    def extract(self, img):
        flat = np.clip(np.rint(img), 0, 255).astype(np.uint8).flatten()
        return bits_to_logo(flat[:NB] & 1) > 127.5


class BlockDCTScheme(Scheme):
    """8x8 block DCT; bit = sign of (c[4,3] - c[3,4]) forced to differ by >= delta (Koch-Zhao style)."""
    name = "Baseline: 8x8 block DCT"
    blind = True
```

```
    needs = "payload length only"

    def __init__(self, bits, delta=24.0):
        self.bits = np.asarray(bits[:NB], dtype=np.uint8)
        self.delta = delta
        n = 8
        k = np.arange(n)[:, None]; i = np.arange(n)[None, :]
        c = np.sqrt(2.0 / n) * np.cos(np.pi * (2 * i + 1) * k / (2 * n)); c[0] /= np.sqrt(2)
        self.C = c

    def embed(self, host):
        out = host.astype(np.float64).copy()
        H, W = host.shape
        idx = 0
        for by in range(0, H, 8):
            for bx in range(0, W, 8):
                if idx >= NB:
                    break
                blk = out[by:by + 8, bx:bx + 8]
                d = self.C @ blk @ self.C.T
                a, b = d[4, 3], d[3, 4]
                m = (a + b) / 2
                if self.bits[idx]:
                    d[4, 3], d[3, 4] = m + self.delta / 2, m - self.delta / 2
                else:
                    d[4, 3], d[3, 4] = m - self.delta / 2, m + self.delta / 2
                out[by:by + 8, bx:bx + 8] = self.C.T @ d @ self.C
                idx += 1
        return np.clip(out, 0, 255)

    def extract(self, img):
        H, W = img.shape
        bits = []
        for by in range(0, H, 8):
            for bx in range(0, W, 8):
                if len(bits) >= NB:
                    break
                d = self.C @ img[by:by + 8, bx:bx + 8] @ self.C.T
                bits.append(1 if d[4, 3] > d[3, 4] else 0)
        return bits_to_logo(bits) > 127.5


class BlindWMScheme(Scheme):
    name = "blind-watermark 0.4.4 (block DWT-DCT-SVD)"
    blind = True
    needs = "two integer passwords + payload length"

    def __init__(self, bits):
        from blind_watermark import WaterMark
        self.WaterMark = WaterMark
        self.bits = np.asarray(bits[:NB], dtype=bool)

    def embed(self, host):
        bwm = self.WaterMark(password_img=1, password_wm=1, processes=1)
        bwm.read_img(img=np.dstack([host.astype(np.uint8)] * 3))
        bwm.read_wm(self.bits, mode="bit")
        e = bwm.embed()
        return np.clip(np.rint(e), 0, 255)          # 3-channel (YUV-domain embedding), kept as 3 channels

    def extract(self, img3):
        bwm = self.WaterMark(password_img=1, password_wm=1, processes=1)
        ex = bwm.extract(embed_img=np.clip(np.rint(img3), 0, 255).astype(np.uint8), wm_shape=NB, mode="bit")
        return bits_to_logo(np.asarray(ex) > 0.5) > 127.5


def channelwise(fn, img):
    """Apply a greyscale attack to each channel of a (H,W,3) image."""
    if img.ndim == 2:
        return fn(img)
    return np.stack([fn(img[..., c]) for c in range(img.shape[2])], axis=-1)


def luma(img):
    return img if img.ndim == 2 else cv2.cvtColor(img.astype(np.float32),
cv2.COLOR_BGR2GRAY).astype(np.float64)


def part_b(ids, attacks):
    logo = make_watermark_logo()
    bits = (logo.flatten() > 127)[:NB]
    ref_logo = bits_to_logo(bits)
    rows = []
    for k, iid in enumerate(ids):
```

```
        host = load(iid)
        suite = make_attack_suite(777 + k)
        schemes = [OursScheme(), BlindWMScheme(bits), LSBScheme(bits), BlockDCTScheme(bits)]
        for sch in schemes:
            try:
                t0 = time.perf_counter(); w = sch.embed(host); t_emb = (time.perf_counter() - t0) * 1e3
                wq = np.clip(np.rint(w), 0, 255)
                p = psnr(host, luma(wq))
            except Exception as exc:  # noqa: BLE001
                rows.append(dict(image_id=iid, scheme=sch.name, attack="(embed)", status=f"error: {exc}"))
                continue
            for an in attacks:
                row = dict(image_id=iid, scheme=sch.name, attack=an, psnr_db=p, t_embed_ms=t_emb, status="ok")
                try:
                    att = channelwise(suite[an], wq)
                    t0 = time.perf_counter(); ex = sch.extract(att); row["t_extract_ms"] = (time.perf_counter()
- t0) * 1e3
                    ex_img = ex.astype(np.float64) * 255.0
                    row["ber"] = float(np.mean((ex_img > 127.5) != (ref_logo > 127.5)))
                    a, b = ref_logo.flatten(), ex_img.flatten()
                    den = np.sqrt((a ** 2).sum()) * np.sqrt((b ** 2).sum())
                    row["nc"] = float((a * b).sum() / den) if den else 0.0
                except Exception as exc:  # noqa: BLE001
                    row.update(status=f"error: {type(exc).__name__}: {exc}")
                rows.append(row)
        print("scheme comparison: image", k + 1, "/", len(ids), flush=True)
    return pd.DataFrame(rows)


def tune_delta(ids):
    """Choose the block-DCT strength so that its mean PSNR matches the project's (~36 dB)."""
    logo = make_watermark_logo(); bits = (logo.flatten() > 127)[:NB]
    target = np.mean([psnr(load(i), OursScheme().embed(load(i))) for i in ids])
    best = None
    for d in (8, 12, 16, 20, 24, 28, 32, 40, 48):
        p = np.mean([psnr(load(i), np.rint(BlockDCTScheme(bits, d).embed(load(i)))) for i in ids])
        if best is None or abs(p - target) < abs(best[1] - target):
            best = (d, p)
    return best, target


def main():
    os.makedirs(RES, exist_ok=True)
    meta = pd.read_csv(os.path.join(DATA, "metadata.csv"))
    real = meta[meta.is_synthetic == 0].image_id.tolist()
    rng = np.random.default_rng(20260713)

    # ---- Part A
    comp_ids = list(rng.choice(real, 3, replace=False))
    parts = []
    for iid in comp_ids:
        df, jsz = part_a([load(iid)])
        df["image_id"] = iid
        parts.append(df)
    A = pd.concat(parts)
    Agg = A.groupby(["operation", "implementation"], sort=False).agg(
        median_ms=("median_ms", "median"), q25_ms=("q25_ms", "median"), q75_ms=("q75_ms", "median"),
        includes_process_and_io=("includes_process_and_io", "first"), agreement=("agreement",
"first")).reset_index()
    ref = Agg.groupby("operation", sort=False).median_ms.transform("first")
    Agg["speed_vs_project"] = ref / Agg.median_ms          # >1 = faster than the project's implementation
    Agg.round(4).to_csv(os.path.join(RES, "tech_benchmark_components.csv"), index=False)
    print(Agg.round(3).to_string())

    # ---- Part B
    sample = list(rng.choice(real, 30, replace=False))
    tune_ids = sample[:5]
    (delta, p_d), p_target = tune_delta(tune_ids)
    print("block-DCT delta tuned to", delta, "PSNR", round(p_d, 2), "target", round(p_target, 2))
    BlockDCTScheme.__init__.__defaults__ = (delta,)
    attacks = ["No Attack", "JPEG Q=50", "JPEG Q=20", "Gaussian Noise (var=0.02)", "Median Filter 3x3",
               "Gaussian Blur (sigma=1.5)", "Rotation 5 deg", "Cropping 25%", "Scaling 0.5x"]
    B = part_b(sample, attacks)
    B.to_csv(os.path.join(RES, "tech_benchmark_schemes_raw.csv"), index=False)
    ok = B[B.status == "ok"]
    S = ok.groupby(["scheme", "attack"], sort=False).agg(mean_ber=("ber", "mean"), median_ber=("ber",
"median"),
                                                         mean_nc=("nc", "mean"), psnr_db=("psnr_db", "mean"),
                                                         t_embed_ms=("t_embed_ms", "median"),
t_extract_ms=("t_extract_ms", "median"),
                                                         ber_le_0_10=("ber", lambda x: (x <= 0.10).mean()),
n=("ber", "size")).reset_index()
    S.round(4).to_csv(os.path.join(RES, "tech_benchmark_schemes.csv"), index=False)
```

```
    print(S.round(3).to_string())

    env = {"python": sys.version.split()[0], "numpy": np.__version__, "scipy": scipy.__version__, "scikit-
image": skimage.__version__,
           "opencv": cv2.__version__, "Pillow": __import__("PIL").__version__, "PyWavelets": pywt.__version__,
           "blind-watermark": __import__("importlib.metadata", fromlist=["version"]).version("blind-
watermark"),
           "ImageMagick": (subprocess.run(["convert", "-version"], capture_output=True,
text=True).stdout.splitlines() or [""])[0] if have("convert") else None,
           "ffmpeg": (subprocess.run(["ffmpeg", "-version"], capture_output=True,
text=True).stdout.splitlines() or [""])[0] if have("ffmpeg") else None,
           "cpu_threads_used": 1, "repeats": 9, "warmup": 2, "component_images": comp_ids, "scheme_images":
sample,
           "jpeg_bytes_example": jsz, "blockdct_delta": delta, "blockdct_psnr": p_d, "project_psnr_tune":
p_target}
    json.dump(env, open(os.path.join(RES, "tech_summary.json"), "w"), indent=2, default=str)


if __name__ == "__main__":
    main()
```

A.11 video_demo.py — Exploratory per-frame video test (V1) (173 lines)

```
"""
video_demo.py
=============
EXPLORATORY extension (the submitted project is image-only; video was listed as
"future scope").  Per-frame application of the same DWT-DCT-SVD scheme:

  1. read N frames from a real clip with OpenCV (cv2.VideoCapture)
  2. standardise each frame like the still images (grey, centre-crop, 512x512)
  3. embed the logo into EVERY frame with its own key (non-blind scheme => one
     key per frame; temporal redundancy is NOT exploited)
  4. re-encode the watermarked frames with a lossy codec (FFmpeg: H.264 at three
     CRF values, MJPEG), decode again, extract from every frame
  5. report frames processed, PSNR, NC/BER, bit-rate, timing

This is a feasibility test on two short clips, NOT a video-watermarking benchmark:
it does not test temporal attacks (frame dropping/re-ordering, frame-rate change),
colour video, or hardware encoders.  One naive frame-drop test shows why per-frame
keys need timestamp/index bookkeeping.

    python video_demo.py [--frames 60]
"""

from __future__ import annotations

import os
for _v in ("OPENBLAS_NUM_THREADS", "OMP_NUM_THREADS", "MKL_NUM_THREADS"):
    os.environ.setdefault(_v, "1")

import argparse
import json
import subprocess
import tempfile
import time
import urllib.request

import cv2
import numpy as np
import pandas as pd
from PIL import Image

from logo import make_watermark_logo
from metrics import bit_error_rate, normalized_correlation, psnr
from watermark_core import embed_watermark, extract_watermark

HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.abspath(os.path.join(HERE, "..", ".."))
RES = os.path.join(ROOT, "results")
CACHE = os.path.join(ROOT, "data", "source_videos")
CLIPS = {
    "vtest.avi": "opencv/opencv/4.x/samples/data/vtest.avi",
    "Megamind.avi": "opencv/opencv/4.x/samples/data/Megamind.avi",
}
SIZE = 512
CODECS = {
    "lossless (FFV1)":  ["-c:v", "ffv1"],
    "H.264 CRF 18":     ["-c:v", "libx264", "-crf", "18", "-pix_fmt", "yuv420p"],
    "H.264 CRF 23":     ["-c:v", "libx264", "-crf", "23", "-pix_fmt", "yuv420p"],
    "H.264 CRF 28":     ["-c:v", "libx264", "-crf", "28", "-pix_fmt", "yuv420p"],
    "MJPEG q=5":        ["-c:v", "mjpeg", "-q:v", "5", "-pix_fmt", "yuvj420p"],
    "MJPEG q=15":       ["-c:v", "mjpeg", "-q:v", "15", "-pix_fmt", "yuvj420p"],
```

```
}


def fetch(name):
    os.makedirs(CACHE, exist_ok=True)
    p = os.path.join(CACHE, name)
    if not os.path.exists(p):
        urllib.request.urlretrieve("https://raw.githubusercontent.com/" + CLIPS[name], p)
    return p


def gray_square(frame_bgr):
    g = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2GRAY)
    s = min(g.shape)
    y0, x0 = (g.shape[0] - s) // 2, (g.shape[1] - s) // 2
    g = g[y0:y0 + s, x0:x0 + s]
    return np.asarray(Image.fromarray(g).resize((SIZE, SIZE), Image.LANCZOS), dtype=np.float64)


def read_frames(path, n):
    cap = cv2.VideoCapture(path)
    if not cap.isOpened():
        raise IOError(f"cannot open {path}")
    fps = cap.get(cv2.CAP_PROP_FPS)
    total = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    w, h = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH)), int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    frames = []
    try:
        while len(frames) < n:
            ok, f = cap.read()
            if not ok:
                break
            frames.append(gray_square(f))
    finally:
        cap.release()
    return frames, dict(fps=fps, total_frames=total, width=w, height=h)


def encode_decode(frames, codec_args, fps, tmp):
    """Pipe 8-bit frames into ffmpeg, then decode them back as raw grey frames."""
    out = os.path.join(tmp, "clip." + ("mkv" if "ffv1" in codec_args else ("avi" if "mjpeg" in codec_args else
"mp4")))
    raw = b"".join(np.clip(np.rint(f), 0, 255).astype(np.uint8).tobytes() for f in frames)
    enc = subprocess.run(["ffmpeg", "-v", "error", "-y", "-f", "rawvideo", "-pix_fmt", "gray", "-s",
f"{SIZE}x{SIZE}",
                          "-r", str(fps), "-i", "-", *codec_args, out], input=raw, capture_output=True)
    if enc.returncode != 0:
        raise RuntimeError(enc.stderr.decode()[:300])
    dec = subprocess.run(["ffmpeg", "-v", "error", "-i", out, "-f", "rawvideo", "-pix_fmt", "gray", "-"],
capture_output=True)
    if dec.returncode != 0:
        raise RuntimeError(dec.stderr.decode()[:300])
    buf = np.frombuffer(dec.stdout, dtype=np.uint8)
    n = buf.size // (SIZE * SIZE)
    decoded = [buf[i * SIZE * SIZE:(i + 1) * SIZE * SIZE].reshape(SIZE, SIZE).astype(np.float64) for i in
range(n)]
    return decoded, os.path.getsize(out)


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--frames", type=int, default=60)
    args = ap.parse_args()
    logo = make_watermark_logo()
    rows, meta = [], {}
    for clip in CLIPS:
        frames, info = read_frames(fetch(clip), args.frames)
        meta[clip] = dict(info, frames_read=len(frames))
        t0 = time.perf_counter()
        marked, keys = [], []
        for f in frames:
            w, k = embed_watermark(f, logo, alpha=0.10)
            k = {kk: vv for kk, vv in k.items() if kk not in ("U", "Vt")}   # extraction does not need U, V^T
            marked.append(np.clip(np.rint(w), 0, 255)); keys.append(k)
        t_embed = (time.perf_counter() - t0) / len(frames)
        fps = info["fps"] if info["fps"] and info["fps"] > 0 else 25
        with tempfile.TemporaryDirectory() as tmp:
            for codec, args_ in CODECS.items():
                row = dict(clip=clip, codec=codec, frames_in=len(frames), status="ok")
                try:
                    dec_wm, size_wm = encode_decode(marked, args_, fps, tmp)
                    dec_ref, size_ref = encode_decode(frames, args_, fps, tmp)
                    n = min(len(dec_wm), len(frames))
```

```
                    row["frames_decoded"] = len(dec_wm)
                    t0 = time.perf_counter()
                    ncs, bers = [], []
                    for i in range(n):
                        ex = extract_watermark(dec_wm[i], keys[i])
                        ncs.append(normalized_correlation(logo, ex)); bers.append(bit_error_rate(logo, ex))
                    row["t_extract_ms_per_frame"] = (time.perf_counter() - t0) / n * 1e3
                    row["psnr_wm_vs_orig_db"] = float(np.mean([psnr(frames[i], dec_wm[i]) for i in range(n)]))
                    row["psnr_codec_only_db"] = float(np.mean([psnr(frames[i], dec_ref[i]) for i in range(n)]))
                    row["nc_mean"] = float(np.mean(ncs)); row["nc_min"] = float(np.min(ncs))
                    row["frames_nc_ge_0_75"] = int(np.sum(np.array(ncs) >= 0.75)); row["ber_mean"] =
float(np.mean(bers))
                    row["size_wm_kb"] = size_wm / 1024; row["size_ref_kb"] = size_ref / 1024
                    # naive frame-drop test: remove every 10th frame, extract with index-aligned keys
                    if codec == "lossless (FFV1)":
                        keep = [i for i in range(len(dec_wm)) if i % 10 != 9]
                        naive = [normalized_correlation(logo, extract_watermark(dec_wm[j], keys[i])) for i, j
in enumerate(keep)]
                        aware = [normalized_correlation(logo, extract_watermark(dec_wm[j], keys[j])) for j in
keep]
                        row["framedrop_nc_naive_index"] = float(np.mean(naive))
                        row["framedrop_nc_with_frame_ids"] = float(np.mean(aware))
                except Exception as exc:  # noqa: BLE001
                    row.update(status=f"error: {type(exc).__name__}: {exc}")
                rows.append(row)
        meta[clip]["embed_ms_per_frame"] = t_embed * 1e3
        meta[clip]["embed_fps_single_thread"] = 1 / t_embed
        print(clip, meta[clip], flush=True)
    df = pd.DataFrame(rows)
    df.to_csv(os.path.join(RES, "video_demo_results.csv"), index=False)
    json.dump(meta, open(os.path.join(RES, "video_demo_meta.json"), "w"), indent=2)
    pd.set_option("display.width", 250)
    print(df.round(3).to_string())


if __name__ == "__main__":
    main()
```

A.12 make_diagram.py — Architecture diagrams (99 lines)

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch, FancyArrowPatch
import os

RESULTS_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "..", "..", "visualizations")


def box(ax, xy, w, h, text, fc="#eaf2f8", ec="#2874a6", fontsize=9.5):
    x, y = xy
    p = FancyBboxPatch((x, y), w, h, boxstyle="round,pad=0.02,rounding_size=0.02",
                       fc=fc, ec=ec, linewidth=1.6)
    ax.add_patch(p)
    ax.text(x + w / 2, y + h / 2, text, ha="center", va="center",
            fontsize=fontsize, wrap=True)


def arrow(ax, p1, p2, color="#333333"):
    a = FancyArrowPatch(p1, p2, arrowstyle="-|>", mutation_scale=14,
                        color=color, linewidth=1.5)
    ax.add_patch(a)


def embedding_diagram():
    fig, ax = plt.subplots(figsize=(11, 4.2))
    ax.set_xlim(0, 11); ax.set_ylim(0, 4.2); ax.axis("off")
    ax.set_title("Watermark Embedding Pipeline", fontsize=13, fontweight="bold", pad=16)

    steps = [
        "Host Image\n(512x512, grayscale)",
        "2D DWT\n(Haar, 1-level)\n-> LL sub-band",
        "2-D DCT\nof LL sub-band\n(global)",
        "SVD\nA = U.S.V^T",
        "S' = S + alpha*Sw\n(Sw: singular\nvalues of logo)",
        "Inverse DCT\n+ Inverse DWT",
        "Watermarked\nImage",
    ]
    n = len(steps)
    w, h = 1.35, 1.5
    gap = (11 - n * w) / (n + 1)
    x = gap
    y = 1.1
```

```
    centers = []
    for s in steps:
        box(ax, (x, y), w, h, s, fontsize=9.2)
        centers.append((x, x + w))
        x += w + gap
    for i in range(n - 1):
        arrow(ax, (centers[i][1], y + h / 2), (centers[i + 1][0], y + h / 2))

    # watermark input branch
    box(ax, (centers[4][0] - 0.1, y + h + 0.55), 1.5, 0.85, "Watermark\nLogo (64x64)", fc="#fdebd0",
ec="#b9770e", fontsize=9.2)
    arrow(ax, (centers[4][0] + 0.6, y + h + 0.55), (centers[4][0] + 0.6, y + h), color="#b9770e")

    plt.tight_layout()
    plt.savefig(os.path.join(RESULTS_DIR, "diagram_embedding.png"), dpi=170)
    plt.close(fig)


def extraction_diagram():
    fig, ax = plt.subplots(figsize=(11, 4.0))
    ax.set_xlim(0, 11); ax.set_ylim(0, 4.0); ax.axis("off")
    ax.set_title("Watermark Extraction Pipeline (non-blind, key-based)", fontsize=13, fontweight="bold",
pad=16)

    steps = [
        "Received Image\n(possibly attacked:\nJPEG / noise / crop /\nrotate / scale ...)",
        "2D DWT\n-> LL sub-band",
        "2-D DCT\n+ SVD -> S_att",
        "Sw' = (S_att - S_orig)\n/ alpha",
        "Reconstruct:\nUw . Sw' . Vw^T",
        "Extracted\nWatermark",
    ]
    n = len(steps)
    w, h = 1.55, 1.5
    gap = (11 - n * w) / (n + 1)
    x = gap
    y = 0.9
    centers = []
    for s in steps:
        box(ax, (x, y), w, h, s, fontsize=9.2, fc="#eafaf1", ec="#1e8449")
        centers.append((x, x + w))
        x += w + gap
    for i in range(n - 1):
        arrow(ax, (centers[i][1], y + h / 2), (centers[i + 1][0], y + h / 2))

    box(ax, (centers[3][0] - 0.3, y + h + 0.5), 2.0, 0.75,
        "Embedding Key\n{S_orig, Uw, Vw, alpha, shapes}", fc="#f9ebea", ec="#943126", fontsize=8.6)
    arrow(ax, (centers[3][0] + 0.7, y + h + 0.5), (centers[3][0] + 0.7, y + h), color="#943126")

    plt.tight_layout()
    plt.savefig(os.path.join(RESULTS_DIR, "diagram_extraction.png"), dpi=170)
    plt.close(fig)


if __name__ == "__main__":
    embedding_diagram()
    extraction_diagram()
    print("done")
```